Jenkins Interview Questions & Answers

How to use this handbook

This handbook answers all 72 unique questions from the source list of 80, in the original order. Eight repeated questions were merged into their first occurrence; the appendix shows exactly which ones.

Each answer follows the same pattern, so you can scan fast:

- **Short answer** — your first 2–4 sentences in the interview. Say this first.
- **Explanation** — the concept in plain words.
- **Hands-on experience** — how you used it in a real project. Speak this part in first person.
- **Example** — a Jenkinsfile snippet, command or flow.
- **Interview tip** — what the interviewer is really checking.

Simple questions skip sections they don't need.

The reference project used in the answers

Most hands-on answers use one realistic project, so your stories stay consistent across the interview:

Developer pushes code → GitHub webhook → Jenkins pipeline → Checkout → Maven build → Unit tests → SonarQube + Quality Gate → Docker build → Trivy scan → Push image to Nexus/ECR → Update Kubernetes manifest repo → ArgoCD syncs to EKS → Monitoring with Prometheus/Grafana.

- **Application:** Java Spring Boot microservices built with Maven (one Python service as well).
- **Jenkins setup:** Jenkins controller on an EC2 instance, builds running on Linux agents and Kubernetes pod agents.
- **Environments:** Dev, QA/Staging and Production on AWS EKS.
- **Security:** Credentials in Jenkins Credentials store, role-based access with the Role-based Authorization Strategy plugin, SSO for login.

> **Tip for the whole interview:** Keep one project story and reuse it. Interviewers trust a candidate whose answers connect to the same pipeline again and again.

Part 1 — Pipelines, Credentials & Quality Gates

1. What is Pipeline as Code in Jenkins, and why is it useful?

Short answer: Pipeline as Code means we write the whole CI/CD process in a file called Jenkinsfile and keep it in the Git repository with the application code. Jenkins reads

that file and runs the pipeline. So the pipeline is versioned, reviewed and changed just like normal code.

Explanation

- Earlier, jobs were created by clicking in the Jenkins UI (Freestyle jobs). That was hard to track and easy to break.
- With Pipeline as Code, the build, test, scan and deploy steps live in the Jenkinsfile.
- Every change to the pipeline goes through a pull request, so we have history, review and easy rollback.
- If Jenkins is rebuilt, the pipeline logic is not lost, because it is in Git.
- The same idea extends to Shared Libraries (common code for many pipelines) and Jenkins Configuration as Code (JCasC) for Jenkins itself.

Hands-on experience: In my project, every microservice repository had its own Jenkinsfile at the root. When a developer wanted to add a new stage, for example a Trivy scan, they raised a pull request on the Jenkinsfile. The DevOps team reviewed it, and once merged, every new build used the new stage automatically.

Example

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh'mvn clean package -DskipTests'
            }
        }
        stage('Test') {
        steps {
                sh'mvn test'
            }
        }
    }
}
```

In the job configuration, we select **Pipeline script from SCM**, give the Git URL and branch, and set the script path to Jenkinsfile.

> **Interview tip:** Mention the three benefits clearly: version control, code review, and easy recovery. Saying "Jenkinsfile in Git, reviewed via PR" shows real practice.

2. What is the difference between Declarative and Scripted pipelines?

Short answer: Both are written in Groovy and both run on the same pipeline engine. Declarative pipeline has a fixed, simple structure starting with `pipeline { }`. Scripted pipeline starts with `node { }` and gives full Groovy programming freedom. For most projects, Declarative is the recommended choice.

Explanation

Point	Declarative	Scripted
Starts with	`pipeline { }`	`node { }`
Structure	Fixed blocks: `agent`, `stages`, `steps`, `post`	Free-form Groovy code
Learning curve	Easy to read and write	Needs Groovy knowledge
Error checking	Syntax validated before the run starts	Errors found while running
Built-in features	`post`, `when`, `options`, `parameters`, `environment`, restart from stage	You code these yourself with try/catch
Best for	Most CI/CD pipelines	Very dynamic or complex logic

Inside Declarative, we can still use a `script { }` block when we need some Groovy logic, like a loop or an `if` condition.

Hands-on experience: In my project, all application pipelines were Declarative because the team could read them easily. Where we needed complex logic, for example looping over a list of microservices, we used a small `script { }` block or moved that logic into a Shared Library.

Example

Declarative:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps { sh 'mvn clean package' }
        }
    }
    post {
        failure { echo 'Build failed' }
    }
}
```

Scripted:

```
node {
    try{
        stage('Build') {
            sh'mvn clean package'
        }
    }catch(err) {
        echo'Build failed'
        throw err
    }
}
```

Notice how Declarative uses `post { failure { } }`, while Scripted needs try/`catch` for the same thing.

> **Interview tip:** Say which one you used and why. "Declarative for readability and built-in features, `script` blocks or Shared Libraries for complex logic" is a strong answer.

3. What stages do you use in your CI/CD pipeline? Explain each stage.

Short answer: Our pipeline had these stages: Checkout, Build, Unit Test, SonarQube Analysis, Quality Gate, Docker Build, Trivy Scan, Push Image, and Deploy. Deployment was done through ArgoCD by updating the image tag in a GitOps repository. Every stage acts as a gate, so bad code stops early.

Explanation and hands-on experience

1. **Checkout** — Jenkins pulls the code from GitHub using a stored credential.
2. **Build** — `mvn clean package` compiles the Java code and creates the JAR file.
3. **Unit Test** — `mvn test` runs JUnit tests. We publish the results with the `junit` step so failures show in Jenkins.

4. **SonarQube Analysis** — checks code quality: bugs, code smells, duplication and test coverage.
5. **Quality Gate** — waits for SonarQube's result. If the gate fails, the pipeline stops.
6. **Docker Build** — builds the image and tags it with the build number or Git commit ID, never only `latest`.
7. **Trivy Scan** — scans the image for vulnerabilities. We failed the build on CRITICAL issues.
8. **Push Image** — pushes the image to the registry (Nexus or AWS ECR).
9. **Update Manifest / Deploy** — Jenkins updates the image tag in the Kubernetes manifest repo. ArgoCD detects the change and deploys to the cluster.
10. **Post actions** — Slack or email notification, workspace cleanup.

For production, we added a **manual approval** stage before the deploy.

Example

```
pipeline {
    agent { label 'linux-docker' }
    environment {
        IMAGE = "123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-
service"
        TAG   = "${env.BUILD_NUMBER}"
    }
    stages {
        stage('Checkout')     { steps { checkout scm } }
        stage('Build')        { steps { sh 'mvn clean package -DskipTests' } }
        stage('Unit Test') {
            steps { sh 'mvn test' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        stage('SonarQube') {
            steps {
                withSonarQubeEnv('sonarqube') { sh 'mvn sonar:sonar' }
            }
        }
        stage('Quality Gate') {
            steps {
                timeout(time: 10, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }
        stage('Docker Build') { steps { sh 'docker build -t $IMAGE:$TAG .'}
}
```

```
        stage('Trivy Scan') {
            steps { sh 'trivy image --exit-code 1 --severity CRITICAL
$IMAGE:$TAG' }
        }
        stage('Push Image') { steps { sh 'docker push $IMAGE:$TAG' } }
    }
    post {
        always { cleanWs() }
    }
}
```

The ECR login step and the GitOps update step are shown in Q60 and Q61.

> **Interview tip:** Explain the *why* of each stage, not just the name. For example: "We scan the image with Trivy before pushing, so a vulnerable image never reaches the registry."

4. How do you trigger a Jenkins pipeline automatically when code is pushed to GitHub?

Short answer: We use a GitHub webhook. When a developer pushes code, GitHub sends an HTTP POST to Jenkins, and Jenkins starts the pipeline immediately. This is better than polling, because it is instant and does not load the Git server.

Explanation

- Install the **GitHub plugin** in Jenkins (for Multibranch, the **GitHub Branch Source** plugin).
- In the Jenkins job, enable **GitHub hook trigger for GITScm polling**.
- In GitHub, go to **Repository → Settings → Webhooks → Add webhook**.
- Payload URL: `https://jenkins.example.com/github-webhook/` (the trailing slash matters).
- Content type: `application/json`. Event: **Just the push event** (add pull requests if needed).
- Jenkins must be reachable from GitHub. If Jenkins is in a private network, we expose only the webhook path through a load balancer or use a relay.

Hands-on experience: In my project, every push to a feature branch or main triggered the Multibranch pipeline through a webhook. Once, builds stopped triggering. I checked **GitHub → Webhooks → Recent Deliveries** and saw a 403 response. The cause was a security group change on the Jenkins load balancer. After allowing GitHub's IP ranges again, the triggers worked.

Example

In a Declarative pipeline (non-multibranch job):

```
pipeline {
    agent any
    triggers {
        githubPush()
    }
    stages {
        stage('Build') { steps { sh 'mvn clean package' } }
    }
}
```

The job must run once manually so Jenkins registers the trigger from the Jenkinsfile.

> **Interview tip:** Mention the troubleshooting place — GitHub's "Recent Deliveries" tab shows the response code. It proves hands-on experience.

5. How do you store credentials securely in Jenkins, and where are they stored?

Short answer: We store credentials in the Jenkins Credentials store (**Manage Jenkins → Credentials**), never in the Jenkinsfile. Jenkins encrypts them and saves them under `JENKINS_HOME`. In the pipeline, we refer to them only by their ID, and Jenkins masks the values in the console log.

Explanation

- **Credential types:** Username with password, Secret text (API tokens), Secret file (kubeconfig), SSH username with private key, Certificate, and AWS credentials (with the AWS Credentials plugin).
- **Where on disk:** global credentials are in `$JENKINS_HOME/credentials.xml`. They are encrypted using keys in `$JENKINS_HOME/secrets/` (like `master.key` and `hudson.util.Secret`). Folder-scoped credentials are stored in that folder's config.
- **Scope:** *Global* is for jobs; *System* is only for Jenkins itself (for example, agent connections). We can also create credentials at **folder level**, so only that team's jobs can use them.
- **Usage:** with `withCredentials` or the `credentials()` helper in the `environment` block.

Hands-on experience: In my project, I created folder-level credentials for each team, like `nexus-creds`, `sonar-token` and `github-token`. The payments team's jobs could not see the orders team's secrets. I also made sure the `secrets/` folder was included in our

Jenkins backup, because without it the encrypted credentials cannot be decrypted after a restore.

Example

```
pipeline {
    agent any
    stages {
        stage('Push to Nexus') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'nexus-
creds',
                                                  usernameVariable:
'NEXUS_USER',
                                                  passwordVariable:
'NEXUS_PASS')]) {
                    sh'mvndeploy-ssettings.xml-Dnexus.user=$NEXUS_USER -
Dnexus.pass=$NEXUS_PASS'
                }
            }
        }
    }
}
```

Use single quotes in `sh` so the shell expands the variable. Double quotes make Groovy insert the secret into the command string, and Jenkins warns about it.

> **Interview tip:** Mention encryption, folder scoping, log masking and the single-quote rule. Those four points show security awareness.

6. How do you manage secrets in a CI/CD pipeline?

Short answer: We never hardcode secrets in code, Jenkinsfiles or Docker images. CI secrets live in the Jenkins Credentials store or in an external vault like AWS Secrets Manager or HashiCorp Vault. Application secrets at runtime come from Kubernetes Secrets, ideally synced from the external vault. We follow least privilege and rotate secrets regularly.

Explanation

- **Build-time secrets** (Git token, registry password, Sonar token): Jenkins Credentials, used via `withCredentials`.
- **Cloud access:** prefer IAM roles over access keys. For example, a Jenkins agent on EC2 uses an **instance profile**, and pods on EKS use **IRSA**. No long-lived keys to leak.

- **Runtime secrets** (DB password): AWS Secrets Manager or Vault, synced into Kubernetes using **External Secrets Operator** or the Secrets Store CSI driver.
- **GitOps repo:** never commit plain Kubernetes Secrets. Use Sealed Secrets or External Secrets instead.
- **Hygiene:** mask secrets in logs, rotate them, restrict who can view or update credentials, and scan code for leaked secrets (for example with Gitleaks or Trivy's secret scanner).

Hands-on experience: In my project, Jenkins agents ran on EC2 with an IAM role, so pushing to ECR needed no access keys. The database password lived in AWS Secrets Manager, and External Secrets Operator created the Kubernetes Secret in the namespace. When we rotated the password, the pod picked up the new value after a restart, with no change to the pipeline.

Example

```
environment {
    SONAR_TOKEN = credentials('sonar-token')    // Secret text, masked in logs
}
```

> **Interview tip:** Separate build-time secrets from runtime secrets. Many candidates mix them up. Saying "IAM roles instead of access keys" is a big plus.

7. What is a Quality Gate?

Short answer: A Quality Gate is a set of pass/fail conditions in SonarQube. For example: no new critical bugs, test coverage on new code above 80%, and duplication below 3%. If the code does not meet these conditions, the gate fails, and we stop the pipeline so poor-quality code does not move forward.

Explanation

- SonarQube analyses the code and calculates metrics: bugs, vulnerabilities, code smells, coverage, duplication, security hotspots.
- The Quality Gate compares these metrics against thresholds.
- The default gate, **"Sonar way"**, focuses on **new code**. This lets teams improve step by step without fixing all old issues first.
- The result is either **Passed** or **Failed**. Jenkins reads this result.

Hands-on experience: In my project, we created a custom Quality Gate: coverage on new code at least 80%, zero new blocker or critical issues, and all new security hotspots reviewed. If a developer's change dropped coverage, the pipeline failed at the Quality Gate stage and the PR could not be merged.

> **Interview tip:** Mention "new code" conditions. It shows you understand how teams adopt SonarQube on existing projects.

8. How does SonarQube fail the Jenkins pipeline when the Quality Gate fails?

Short answer: SonarQube itself does not stop Jenkins. Jenkins sends the analysis, SonarQube processes it in the background, and then SonarQube calls a webhook back to Jenkins with the Quality Gate result. The `waitForQualityGate` step receives that result, and with `abortPipeline: true` it marks the pipeline as failed.

Explanation — the flow

1. `withSonarQubeEnv('sonarqube')` injects the SonarQube URL and token.
2. `mvn sonar:sonar` uploads the analysis report. SonarQube creates a background task.
3. SonarQube computes the Quality Gate.
4. SonarQube calls the webhook `https://jenkins.example.com/sonarqube-webhook/`.
5. `waitForQualityGate` receives the status. If it is not OK and `abortPipeline: true`, the build fails.

Setup needed

- Install the **SonarQube Scanner** plugin in Jenkins.
- Add the SonarQube server in **Manage Jenkins → System** with a token credential.
- In SonarQube: **Administration → Configuration → Webhooks**, add the Jenkins URL ending in `/sonarqube-webhook/`.

Hands-on experience: Once, our pipeline was stuck at the Quality Gate stage until the timeout. The analysis was fine. The real problem was that the SonarQube webhook was pointing to an old Jenkins URL after a migration. After fixing the webhook URL, results came back in seconds.

> **Interview tip:** The webhook is the key point. If you say "SonarQube calls back Jenkins through a webhook," the interviewer knows you have configured it yourself.

9. What does `waitForQualityGate` do in a Jenkins pipeline?

Short answer: `waitForQualityGate` pauses the pipeline until SonarQube sends the Quality Gate result. If the gate fails and `abortPipeline: true` is set, it fails the pipeline. We always wrap it in a `timeout` so the pipeline does not wait forever.

Explanation

- It comes from the SonarQube Scanner plugin.

- It uses the analysis task from the previous `withSonarQubeEnv` block in the same pipeline.
- It does not use an executor heavily while waiting, but it still holds the build open, so a timeout is important.
- `abortPipeline: true` → fail the build on a failed gate.
- `abortPipeline: false` → return the result and let you decide (see Q65).

Example

```
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('sonarqube') {
            sh 'mvn sonar:sonar -Dsonar.projectKey=orders-service'
        }
    }
}
stage('Quality Gate') {
    steps {
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}
```

- `withSonarQubeEnv('sonarqube')` — 'sonarqube' is the server name configured in Jenkins.
- `timeout(...)` — if no webhook response comes in 5 minutes, the stage fails instead of hanging.

> **Interview tip:** Always mention the `timeout` wrapper and the webhook dependency. These two are what experienced engineers talk about.

10. If a Jenkins job fails, how do you troubleshoot it?

Short answer: First I open the console output and find the exact stage and error. Then I decide whether it is a code problem, a configuration problem (credentials, environment variables), or an infrastructure problem (agent, disk, network). I fix the root cause, re-run the job, and add a check so it doesn't happen again.

My step-by-step approach

1. **Read the console output** from the bottom up. Find the first real error, not the last line.
2. **Find the failed stage** using Stage View or Pipeline Graph View.

3. **Check what changed** — a new commit, a plugin update, a changed credential, a new base image? Compare with the last successful build.
4. **Check credentials and config** — is the credential ID correct and in scope? Are environment variables set?
5. **Check the agent** — is the node online? Is disk full? Is Docker running? Is Java/Maven the right version?
6. **Check connectivity** — can the agent reach GitHub, Nexus, SonarQube, the registry or the cluster?
7. **Check tool logs** — Maven test reports, `docker build` output, `kubectl describe pod` and `kubectl logs` for deploy failures.
8. **Reproduce** — run the same command on the agent manually, or use **Replay** to test a Jenkinsfile change without committing.
9. **Fix and re-run** — use **Restart from Stage** in Declarative pipelines to save time.
10. **Prevent** — add validation, notifications, better error messages, or monitoring.

Hands-on experience: One morning, many builds failed with `No space left on device`. The console log pointed to the Docker build stage. I checked the agent and found old Docker images filling the disk. I cleaned them with `docker system prune -af`, re-ran the builds, and then added a scheduled cleanup job and a disk-space alert so it would not happen again.

> **Interview tip:** Always end with prevention. Interviewers want to hear that you fix the root cause, not just re-run the build.

11. If a pipeline is slow — for example, it takes 45 minutes — how do you optimize it?

(Merged with source Q51: "How do you shorten a pipeline that takes 45 minutes?")

Short answer: First I measure which stages take the most time. Then I run independent stages in parallel, cache dependencies and Docker layers, use faster or more agents, and only build what changed. In my project, these steps brought a 40-minute pipeline down to around 12 minutes.

Step-by-step approach

1. **Measure first.** Use Stage View or Pipeline Graph View to see the time per stage. Don't guess.
2. **Parallelize.** Unit tests, SonarQube analysis and dependency checks can run side by side.

3. **Cache dependencies.** Keep the Maven `~/.m2` repository on the agent or a persistent volume, and use a Nexus proxy so downloads are local.
4. **Use Docker layer caching.** Copy `pom.xml` and download dependencies before copying the source code, so dependency layers are reused. Use multi-stage builds.
5. **Shallow clone.** Clone with `depth: 1` and no tags for large repositories.
6. **Build only what changed.** In a monorepo, `use when { changeset 'orders-service/**' }` so unchanged services are skipped.
7. **Faster Maven.** `mvn -T 1C` for parallel module builds, `-o` (offline) when the cache is warm, and split slow integration tests into a separate nightly job.
8. **Better agents.** More CPU/memory, or Kubernetes pod agents that scale out when the queue grows.
9. **Remove waste.** Duplicate builds, unneeded archive steps, long `sleep` calls.
10. **Avoid queue time.** Check if builds are waiting for free executors — sometimes the build is fast but the wait is slow.

Hands-on experience: Our pipeline took about 40 minutes. Stage timings showed that Maven downloads took 10 minutes and tests ran one after another. I added a Nexus proxy and persistent `.m2` cache, ran unit tests and SonarQube in parallel, reordered the Dockerfile for layer caching, and moved slow integration tests to a nightly job. The pipeline dropped to about 12 minutes.

Example

```
stage('QualityChecks') {
    parallel {
        stage('UnitTests') {
            steps{sh 'mvn test' }
        }
        stage('SonarQube') {
            steps {
                withSonarQubeEnv('sonarqube') { sh 'mvn sonar:sonar -
  DskipTests' }
             }
        }
    }
}
```

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline    # cached unless pom.xml changes
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:17-jre
COPY --from=build /app/target/*.jar /app/app.jar
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

> **Interview tip:** Start with "I measure first." Then give numbers: before and after. Numbers make the story believable.

Part 2 — Pipeline Building Blocks & Design

12. What is a Jenkins Shared Library, and why do you use it?

Short answer: A Shared Library is a Git repository of reusable Groovy code that many pipelines can use. Instead of copying the same 200-line Jenkinsfile into 30 microservice repos, we write the common steps once in the library and call them from each Jenkinsfile. A fix in the library updates every pipeline.

Explanation

- **Folder structure of the library repo:**
  - `vars/` — global steps you call directly, like `buildMavenApp()` or `dockerBuildPush()`. Each file has a `call()` method.
  - `src/` — Groovy classes for more complex logic.
  - `resources/` — non-Groovy files, like templates or shell scripts, loaded with `libraryResource`.
- **Configure it:** Manage Jenkins → System → **Global Pipeline Libraries**. Give a name, the Git URL and a default version (branch or tag).
- **Use it:** add `@Library('devops-lib') _` at the top of the Jenkinsfile.
- **Pin versions:** use a tag like `@Library('devops-lib@v1.4') _` so a library change does not break all pipelines at once.

Hands-on experience: In my project, we had around 25 Java microservices. Each Jenkinsfile was only about 10 lines because it called a shared step named `javaMicroservicePipeline()`. When we added Trivy scanning, I changed one file in the

library, tested it on one service using a feature-branch version of the library, then released a new tag. All services got the Trivy stage without touching their repos.

Example

`vars/dockerBuildPush.groovy` in the library:

```
def call(String imageName, String tag) {
    sh "docker build -t ${imageName}:${tag} ."
    sh "docker push ${imageName}:${tag}"
}
```

Jenkinsfile in the application repo:

```
@Library('devops-lib@v1.4') _

pipeline {
    agent{label'docker' }
    stages {
        stage('Build & Push') {
            steps {
                dockerBuildPush('myregistry/orders-service',
env.BUILD_NUMBER)
            }
        }
    }
}
```

The `_` after `@Library` is needed because the annotation must be attached to something; `_` is a placeholder.

> **Interview tip:** Mention versioning (tags) and testing the library on one service first. That shows you know a library change can break many pipelines at once.

13. What are the default (built-in) environment variables in Jenkins?

Short answer: Jenkins provides built-in variables for every build, like `BUILD_NUMBER`, `JOB_NAME`, `WORKSPACE`, `BUILD_URL` and `NODE_NAME`. In Multibranch pipelines we also get `BRANCH_NAME` and, for pull requests, `CHANGE_ID`. We use them to tag images, name artifacts and send useful notifications.

Common variables

Variable	What it holds	How I used it
`BUILD_NUMBER`	Current build number, e.g. 57	Docker image tag
`BUILD_ID`	Build ID (same as number in current Jenkins)	Unique IDs
`JOB_NAME`	Job name, including folder path	Notifications
`BUILD_URL`	Link to this build	Slack message link
`WORKSPACE`	Path of the workspace on the agent	File paths
`JENKINS_URL`	Jenkins base URL	Links in emails
`NODE_NAME`	Agent where the build runs	Troubleshooting
`BRANCH_NAME`	Branch name (Multibranch only)	Deploy only from `main`
`CHANGE_ID`	Pull request number (Multibranch PR builds)	Comment on the PR
`GIT_COMMIT`	Commit SHA (set by the Git plugin)	Image tag for traceability

You can see the full list at `https://<jenkins-url>/env-vars.html`, or by running `sh 'printenv'` in a pipeline. Build parameters are available as `params.NAME`, and the build status as `currentBuild.result`.

Example

```
stage('DockerBuild') {
    steps {
        script {
            env.IMAGE_TAG = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
        }
        sh'docker build -t orders-service:$IMAGE_TAG .'
    }
}
```

This gives a tag like `57-a1b2c3d`: build number plus short commit ID, so we can trace any running image back to the exact commit.

> **Interview tip:** Name 5–6 variables and say how you used each one. "Image tag = build number + short commit SHA" is a very common real-world answer.

14. What does `agent any` mean in a Jenkins pipeline?

Short answer: `agent any` tells Jenkins to run the pipeline (or stage) on any available node that has a free executor. It is simple, but in production we usually use a specific label instead, because not every agent has the right tools like Docker or Maven.

Explanation

The `agent` directive decides *where* the pipeline runs. Options:

- `agent any` — any available agent.
- `agent none` — no global agent; each stage defines its own.
- `agent { label 'docker' }` — only agents with that label.
- `agent { docker { image 'maven:3.9-eclipse-temurin-17' } }` — run inside a Docker container.
- `agent { kubernetes { yaml '...' } }` — run in a temporary Kubernetes pod.

Hands-on experience: Early in my project, we used `agent any`. Sometimes builds landed on the built-in node (the controller), which slowed Jenkins down and was a security risk. We changed the controller's executors to 0, and used labels like `maven` and `docker` so builds always ran on the right agents.

> **Interview tip:** Say that builds should not run on the controller. Setting the built-in node to 0 executors is a recognized best practice.

15. What is a label in Jenkins?

Short answer: A label is a tag we give to agents, like `linux`, `docker`, `maven` or `windows`. In the pipeline we ask for a label, and Jenkins runs the job only on agents with that label. It is how we send the right work to the right machine.

Explanation

- Set labels in **Manage Jenkins → Nodes → (agent) → Configure → Labels**. One agent can have many labels.
- Label expressions are supported: `agent { label 'linux && docker' }` or `'java17 || java21'`.
- Labels make scaling easy: add a new agent with the same label, and Jenkins starts using it automatically.
- For cloud agents (EC2 or Kubernetes plugin), the label is defined in the cloud template, and Jenkins creates agents on demand for that label.

Hands-on experience: In my project, we had agents labelled `linux-docker` for image builds, `maven-jdk17` for Java builds, and `deploy-prod`, a single hardened agent inside the production network for deployments. Only production pipelines used the `deploy-prod` label.

Example

```
pipeline {
    agent{label 'linux && docker' }
    stages {
        stage('Build Image') {
            steps { sh 'docker build -t app:$BUILD_NUMBER .' }
        }
    }
}
```

> **Interview tip:** Mention label expressions (`&&`, `||`) and that labels let you scale by adding agents without changing pipelines.

16. Can we run stages in parallel in Jenkins?

Short answer: Yes. In Declarative pipelines we use the `parallel` block inside a stage. Independent stages, like unit tests, SonarQube analysis and dependency scanning, run at the same time, which reduces total pipeline time.

Explanation

- Only parallelize stages that don't depend on each other.
- Each parallel branch can run on a different agent.
- `failFast true` stops all other branches as soon as one fails, which saves time.
- For testing many combinations (for example, Java 17 and Java 21), the `matrix` directive creates parallel stages automatically.

Hands-on experience: In my project, unit tests took 8 minutes and SonarQube took 5 minutes. Running them in parallel saved about 5 minutes per build. In a monorepo with three services, I also built all three Docker images in parallel.

Example

```
stage('Checks') {
    failFast true
    parallel {
        stage('Unit Tests') {
            steps { sh 'mvn test' }
        }
        stage('Dependency Scan') {
            steps { sh 'trivy fs --severity HIGH,CRITICAL .' }
        }
        stage('Lint Dockerfile') {
            steps { sh 'hadolint Dockerfile' }
        }
    }
}
```

> **Interview tip:** Add the caution: parallel stages that share the same workspace can clash. Use separate agents or `stash`/`unstash` when needed.

17. Can we run a specific stage on a specific agent?

Short answer: Yes. We define an `agent` inside that stage. A common pattern is `agent none` at the top, and each stage picks its own agent by label or Docker image.

Explanation

- Stage-level `agent` overrides the pipeline-level agent.
- Files are not shared automatically between agents. Use `stash` in one stage and `unstash` in another, or publish to an artifact repository.
- Using `agent none` at top level avoids holding an executor while waiting, for example during an `input` approval.

Hands-on experience: In my project, the build ran on a Maven agent, the image build ran on a Docker agent, and the production deploy ran only on a special agent inside the production VPC. I passed the JAR between stages with `stash` and `unstash`.

Example

```
pipeline {
    agent none
    stages {
        stage('Build') {
            agent { label 'maven-jdk17' }
            steps {
                sh 'mvn clean package -DskipTests'
```

```
                stashname:'app-jar', includes: 'target/*.jar'
            }
        }
        stage('Docker Image') {
            agent{label'linux-docker' }
            steps {
                checkout scm
                unstash 'app-jar'
                sh'dockerbuild -t orders-service:$BUILD_NUMBER .'
            }
        }
    }
}
```

- `stash` saves files from the first agent; `unstash` restores them on the second.
- `checkout scm` in the second stage brings the Dockerfile, because it is a fresh workspace.

> **Interview tip:** Mention stash/unstash. Many candidates forget that each agent has its own workspace.

18. How do you integrate SonarQube with Jenkins?

Short answer: We install the SonarQube Scanner plugin, add the SonarQube server URL and token in Jenkins, configure a webhook in SonarQube back to Jenkins, and then add two stages to the pipeline: analysis with `withSonarQubeEnv`, and the gate check with `waitForQualityGate`.

Setup steps

1. **SonarQube side:** create a project (or let the first analysis create it), generate a token under **My Account → Security**, and add a webhook to `https://jenkins.example.com/sonarqube-webhook/`.
2. **Jenkins credentials:** add the token as a **Secret text** credential, for example `sonar-token`.
3. **Jenkins system config:** Manage Jenkins → System → **SonarQube servers**: name `sonarqube`, URL, and the token credential.
4. **Scanner tool:** for Maven projects, the `sonar-maven-plugin` is used through `mvn sonar:sonar`. For other languages (Python, Node), configure **SonarQube Scanner** under Manage Jenkins → Tools.
5. **Pipeline:** analysis stage, then Quality Gate stage.

Hands-on experience: For Java services, I used `mvn sonar:sonar` with JaCoCo coverage reports so SonarQube could show test coverage. For our Python service, I used the standalone scanner with `sonar-project.properties` and passed the `coverage.xml` from pytest.

Example (Python service)

```
stage('SonarQube Analysis') {
    environment {
        SCANNER_HOME = tool 'sonar-scanner'
    }
    steps {
        withSonarQubeEnv('sonarqube') {
            sh '''
              $SCANNER_HOME/bin/sonar-scanner \
                -Dsonar.projectKey=payments-api \
                -Dsonar.sources=. \
                -Dsonar.python.coverage.reportPaths=coverage.xml
            '''
        }
    }
}
stage('Quality Gate') {

    steps {
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}
```

> **Interview tip:** Mention coverage reports (JaCoCo for Java, coverage.xml for Python). Without them SonarQube shows 0% coverage, which is a common real-world issue.

19. How would you design a CI/CD pipeline for microservices?

Short answer: I keep each microservice independent: its own repo or folder, its own pipeline, its own Docker image and version. A Shared Library holds the common pipeline logic. CI builds, tests, scans and pushes the image. CD is done with GitOps: Jenkins updates the image tag in a manifest repo, and ArgoCD deploys to Kubernetes, environment by environment.

Design points

1. **Repository layout** — one repo per service (or a monorepo with `when { changeset }` so only changed services build).

2. **Multibranch pipelines** — feature branches and PRs get CI checks automatically; only `main` deploys.
3. **Shared Library** — one standard pipeline for all services, so every service gets the same quality and security checks.
4. **CI stages** — checkout, build, unit test, SonarQube + Quality Gate, Docker build, Trivy scan, push with an immutable tag (commit SHA).
5. **CD with GitOps** — a separate manifest repo (Helm or Kustomize) with folders per environment. Jenkins updates the tag; ArgoCD syncs.
6. **Promotion** — Dev automatic → QA after tests → Production after manual approval. The same image moves through all environments, it is never rebuilt.
7. **Agents** — Kubernetes pod agents, so many services can build at the same time.
8. **Safety** — health checks, readiness probes, rolling or canary deployments, and quick rollback by reverting the Git commit.
9. **Observability** — notifications to Slack, metrics in Prometheus/Grafana, logs centralised.

Hands-on experience: In my project, each of our ~25 services had a short Jenkinsfile calling the shared library. A push to `main` built the image, pushed it to ECR with the commit SHA as the tag, and updated `envs/dev/values.yaml` in the GitOps repo. ArgoCD deployed it to the Dev namespace within a minute. Promotion to production was a PR in the GitOps repo, approved by the release manager.

Example flow

Developer push → GitHub webhook → Jenkins Multibranch job → Build/Test → Sonar → Docker build → Trivy → Push to ECR → Update GitOps repo (dev) → ArgoCD sync to EKS → Smoke tests → Promote to QA/Prod via PR + approval.

> **Interview tip:** Say "build once, deploy many." Rebuilding the image per environment is a red flag for interviewers.

20. What are the types of pipelines in Jenkins, and which ones have you implemented?

(Merged with source Q35: "What types of Jenkins pipelines have you implemented?")

Short answer: By syntax, there are two types: **Declarative** and **Scripted**. By job type, the common ones are **Freestyle**, **Pipeline**, **Multibranch Pipeline** and **Organization Folder**. In my project, I mainly used Declarative Multibranch pipelines, plus a few parameterized pipeline jobs for operations tasks.

Explanation

Type	What it is	When to use
Freestyle	UI-configured job, no Jenkinsfile	Simple or legacy tasks
Pipeline	One job running a Jenkinsfile	A single branch or an ops task
Multibranch Pipeline	Creates a job per branch and PR automatically	Application repos (most common)
Organization Folder	Scans a whole GitHub org and creates Multibranch jobs for every repo with a Jenkinsfile	Many repos, less setup
Declarative syntax	`pipeline { }`, structured	Default choice
Scripted syntax	`node { }`, full Groovy	Complex dynamic logic

The older Multi-configuration (Matrix) job type also exists, but today the `matrix` directive inside a Declarative pipeline is the modern way.

Hands-on experience: What I implemented:

- **Multibranch pipelines** for all microservices — CI on every branch and PR, deploy from `main`.
- **Parameterized pipeline jobs** for operations, like a "Rollback" job that takes a service name and image tag.
- **Scheduled pipelines** for nightly integration tests and Docker cleanup.
- **Shared Library** based pipelines so every service followed the same standard.

> **Interview tip:** Answer both angles — syntax types and job types — then list what *you* built. That covers both versions of this question.

Part 3 — Jenkins Basics & Administration

21. What is Jenkins, and why do we use it?

Short answer: Jenkins is an open-source automation server, mainly used to implement CI/CD. It automatically builds, tests, scans and deploys our code whenever developers push changes. In my project, Jenkins ran our whole flow from GitHub to Kubernetes, so releases became fast, repeatable and less error-prone.

Why we use it

- **Automation** — no manual builds or copy-paste deployments.
- **Fast feedback** — developers know within minutes if their change broke something.
- **Huge plugin ecosystem** — Git, Docker, Kubernetes, SonarQube, AWS, Slack, and many more.
- **Pipeline as Code** — the process lives in a Jenkinsfile in Git.
- **Distributed builds** — a controller plus many agents, so it scales with the team.
- **Free and self-hosted** — full control over security and data, which many companies need.

Hands-on experience: In my project, when a developer pushed code to GitHub, a webhook triggered Jenkins. Jenkins checked out the code, built it with Maven, ran SonarQube analysis, built a Docker image, scanned it with Trivy, pushed it to the registry, and updated the manifest so ArgoCD deployed it to Kubernetes.

> **Interview tip:** Keep the definition to one sentence, then move straight into what Jenkins did in your project. That is what the interviewer actually wants.

22. The Jenkins service is down. How will you troubleshoot it?

Short answer: First I check the service status and the Jenkins logs to find why it stopped. The common causes are a Java version problem after an upgrade, a full disk, low memory, a port conflict, a broken plugin, or wrong permissions on `JENKINS_HOME`. I fix the cause, start the service, and confirm it is healthy.

My step-by-step approach

1. **Check the service:**

```
sudo systemctl status jenkins
```

2. **Read the logs:**

```
sudo journalctl -u jenkins -n 200 --no-pager
# older installs may also write to:
sudo tail -n 200 /var/log/jenkins/jenkins.log
```

3. **Check disk space** — a full disk is the most common cause:

```
df -h
sudo du -sh /var/lib/jenkins/* | sort -h | tail
```

4. **Check memory** — look for out-of-memory kills: `free -m` and `dmesg -T | grep -i oom`.

5. **Check Java** — `java -version`. Current Jenkins LTS needs Java 17 or 21. An OS update or a Jenkins upgrade can cause a mismatch.
6. **Check the port** — `sudo ss -tulpn | grep 8080`. Another process may be using it.
7. **Check permissions** — JENKINS_HOME must be owned by the `jenkins` user: `sudo chown -R jenkins:jenkins /var/lib/jenkins`.
8. **Check plugins** — if the log shows a plugin failing after an upgrade, disable it by creating a `<plugin>.jpi.disabled` file in the `plugins` folder, or roll the plugin back.
9. **Start and verify:** `sudo systemctl start jenkins`, then open the UI and run a test job.
10. **Prevent:** disk and memory alerts, log rotation, build discarders, and testing upgrades on a staging Jenkins first.

Hands-on experience: Once Jenkins went down at night. `journalctl` showed that Jenkins could not write to disk. The `jobs` folder had grown to hundreds of GB because old builds were never deleted. I cleaned old builds and archived artifacts, started Jenkins, and then added `buildDiscarder(logRotator(numToKeepStr: '20'))` to all pipelines and a disk alert at 80% usage.

> **Interview tip:** Mention `journalctl -u jenkins` — newer Jenkins packages log to the systemd journal by default. It shows you know the current setup.

23. Jenkins is not accessible in the browser. What could be the reasons?

Short answer: I check from inside to outside: first whether the Jenkins service is running and listening on its port, then the server firewall, then the cloud security group or load balancer, then DNS, the reverse proxy and SSL. Usually it is one of these layers.

Possible reasons and checks

Reason	How I check
Jenkins service is stopped or still starting	`systemctl status jenkins`; `curl -I http://localhost:8080` on the server
Jenkins listens only on localhost	Check `--httpListenAddress` / `JENKINS_LISTEN_ADDRESS` in the service config
OS firewall blocks the port	`sudo ufw status` or `sudo firewall-cmd --list-all`
AWS security group doesn't allow 8080/443 from my IP	EC2 → Security Groups → Inbound rules

Reason	How I check
Public IP changed after EC2 stop/start	Use an Elastic IP or a DNS name behind a load balancer
Reverse proxy (Nginx/ALB) misconfigured	Check Nginx logs, ALB target group health
SSL certificate expired	Browser error; check certificate dates
DNS record wrong	`nslookup jenkins.example.com`
Server overloaded (high CPU/memory)	`top`, `free -m`; Jenkins may be too slow to respond

Hands-on experience: In my project, Jenkins suddenly became unreachable after a server restart. The service was running and `curl localhost:8080` worked. The problem was the EC2 public IP had changed after the stop/start. We moved Jenkins behind an Application Load Balancer with a fixed DNS name and HTTPS, so it never happened again.

> **Interview tip:** The `curl localhost:8080` test is a strong point. It quickly tells you whether the problem is Jenkins itself or the network in front of it.

24. What is the purpose of Poll SCM and Build Periodically?

Short answer: Both are time-based triggers using cron syntax. **Poll SCM** checks Git on a schedule and builds *only if there are new commits*. **Build Periodically** builds on a schedule *whether or not code changed*. Today, for code changes we prefer webhooks; Build Periodically is used for nightly or scheduled jobs.

Explanation

	Poll SCM	Build Periodically
Builds when	New commits are found	The time arrives, always
Load	Repeatedly calls the Git server	Runs the full job every time
Typical use	When webhooks are not possible (Jenkins not reachable from Git)	Nightly tests, cleanup, reports, backups
Pipeline syntax	`pollSCM('H/5 * * * *')`	`cron('H 2 * * *')`

- `H` means "hash": Jenkins picks a consistent but spread-out minute per job, so 100 jobs don't all start at the same second.

- Cron fields: minute, hour, day of month, month, day of week.

Hands-on experience: We used webhooks for application builds. We used **Build Periodically** for a nightly regression test suite at 2 AM, and a weekly job to prune old Docker images on agents. We used **Poll SCM** only for one internal Git server that was in a network GitHub-style webhooks could not reach.

Example

```
pipeline {
    agent any
    triggers {
        cron('H 2 * * 1-5')                 // weekdays around 2 AM
        //pollSCM('H/5 * * * *') // check Git roughly every 5 minutes
    }
    stages {
        stage('Nightly Tests') {
            steps { sh 'mvn verify -Pintegration-tests' }
        }
    }
}
```

> **Interview tip:** Say "webhooks are preferred over polling" and explain the `H` symbol. Both points show depth.

25. How do you set up (enable) a Multibranch Pipeline in Jenkins?

Short answer: We create a new item of type **Multibranch Pipeline**, connect it to the Git repository with credentials, and choose which branches and pull requests to discover. Jenkins scans the repo and creates a separate pipeline job for every branch that has a `Jenkinsfile`. A webhook keeps it updated automatically.

Steps

1. Make sure the **Pipeline: Multibranch** and **GitHub Branch Source** plugins are installed.
2. **New Item → Multibranch Pipeline**, give it a name.
3. **Branch Sources → Add source → GitHub**. Choose credentials (a GitHub App or a personal access token) and the repository URL.
4. **Behaviours:** discover branches, discover pull requests from origin (and from forks, with a trust policy), and optionally filter branches by name with a wildcard or regex.
5. **Build Configuration:** mode "by Jenkinsfile", script path `Jenkinsfile`.
6. **Scan triggers:** add a periodic scan as a backup (for example, once a day). Webhooks handle the real-time events.

7. **Orphaned Item Strategy:** remove jobs for deleted branches after a few days.
8. Save. Jenkins scans the repo and creates jobs per branch and PR.

Hands-on experience: In my project, I used a **GitHub App** as the credential instead of a personal token. It gives higher API rate limits and is not tied to one person's account. That fixed our "API rate limit exceeded" errors during branch scans.

> **Interview tip:** Mention that a branch needs a Jenkinsfile to be picked up, and that a GitHub App is better than a personal token.

26. Where are Jenkins logs stored on Linux, and how do you use them to troubleshoot?

Short answer: There are two kinds of logs. The **Jenkins system log**: on modern systemd installs, read it with `journalctl -u jenkins`; older installs wrote to `/var/log/jenkins/jenkins.log`. The **build logs** are stored inside `JENKINS_HOME`, under each job's `builds/<number>/log` file. We also see logs in the UI.

Where to look

Log	Location
Jenkins service log (systemd)	`journalctl -u jenkins`
Jenkins service log (older packages)	`/var/log/jenkins/jenkins.log`
Build console log	`/var/lib/jenkins/jobs/<job>/builds/<n>/log`
Multibranch build log	`/var/lib/jenkins/jobs/<job>/branches/<branch>/builds/<n>/log`
System log in UI	Manage Jenkins → System Log (add custom log recorders here)
Agent connection log	Manage Jenkins → Nodes → (agent) → Log
Jenkins in Docker	`docker logs <container>`
Jenkins on Kubernetes	`kubectl logs <jenkins-pod> -n jenkins`

`/var/lib/jenkins` is the default `JENKINS_HOME` for Linux package installs.

Hands-on experience: When agents kept disconnecting, the build log only said "agent went offline". I created a **custom log recorder** in Manage Jenkins → System Log for the remoting classes, and checked the agent's own log. It showed the agent process was being killed by the OS for memory. Increasing the agent's memory fixed it.

Useful commands

```
sudo journalctl -u jenkins -f                    # follow live
sudo journalctl -u jenkins --since "1 hour ago"
grep -i "severe\|exception" /var/log/jenkins/jenkins.log
```

> **Interview tip:** Separate system logs from build logs. Interviewers often ask this to see if you know the `JENKINS_HOME` structure.

27. Write a basic Jenkins pipeline and explain its syntax.

Short answer: A Declarative pipeline starts with `pipeline`, then `agent` (where to run), `stages` with one or more `stage` blocks, `steps` inside each stage (what to run), and an optional `post` block (what to do after). We can also add `environment`, `options`, `parameters` and `triggers`.

Example

```
pipeline {
    agent { label 'maven-jdk17' }

    options {
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '20'))
        disableConcurrentBuilds()
    }

    environment {
        APP_NAME = 'orders-service'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }
        stage('Test') {
        steps {
                sh 'mvn test'
```

```
                }
                post {
                      always { junit 'target/surefire-reports/*.xml' }
                }
          }
     }

     post {
          success{echo "${APP_NAME} build ${BUILD_NUMBER} succeeded" }
          failure{echo "${APP_NAME} build ${BUILD_NUMBER} failed" }
          always   {cleanWs() }
     }
}
```

What each part does

- `agent` — runs the pipeline on an agent with the label `maven-jdk17`.
- `options` — stop after 30 minutes, keep only 20 builds, don't run two builds of this job at the same time.
- `environment` — variables available in all stages.
- `stages` / `stage` — logical steps that show up in the Stage View.
- `steps` — the actual commands.
- `checkout scm` — checks out the same repo and commit that triggered the job.
- `junit` — publishes test results, even if tests fail (`always`).
- `post` — runs after the pipeline: `success`, `failure`, `always`, also `unstable`, `changed` and `cleanup`.
- `cleanWs()` — deletes the workspace (needs the Workspace Cleanup plugin).

> **Interview tip:** If asked to write it on a whiteboard, the minimum is `pipeline → agent → stages → stage → steps`. Add `post` and `options` to show production experience.

Part 4 — Deployments, Integrations & Security

28. What is Blue-Green deployment, and how have you implemented it using Jenkins?

Short answer: In Blue-Green deployment we run two identical environments. Blue is live; Green gets the new version. After testing Green, we switch the traffic from Blue to Green in one step. If something goes wrong, we switch back to Blue immediately, so rollback takes seconds and users see almost no downtime.

Explanation

- **Blue** = current production version. **Green** = new version, deployed side by side.
- Test Green using an internal URL or a test service.
- Switch traffic: change the Kubernetes Service selector, the load balancer target group, or the DNS record.
- Keep Blue running for some time as an instant rollback option, then remove it.
- **Trade-off:** you need double resources during the switch, and database changes must be backward compatible with both versions.

Hands-on experience: In my project on EKS, we had two Deployments: `orders-blue` and `orders-green`. The Service `orders-svc` selected pods by the label `version`. The Jenkins pipeline deployed the new image to the idle color, ran smoke tests against it, waited for manual approval, and then patched the Service selector to the new color. Rollback was just patching the selector back.

Example

```
pipeline {
    agent { label 'deploy' }
    parameters {
        string(name: 'IMAGE_TAG', description: 'Image tag to release')
    }
    environment {
        NS = 'prod'
    }
    stages {
        stage('Find idle color') {
            steps {
                script {
                    def live = sh(script: "kubectl get svc orders-svc -n
${env.NS} -o jsonpath='{.spec.selector.version}'",
                                  returnStdout: true).trim()
                    env.LIVE = live
                    env.IDLE = (live == 'blue') ? 'green' : 'blue'
                    echo "Live: ${env.LIVE}, deploying to: ${env.IDLE}"
                }
            }
        }
        stage('Deploy to idle') {
            steps {
                sh '''
                  kubectl set image deployment/orders-$IDLE
orders=myrepo/orders:$IMAGE_TAG -n $NS
                  kubectl rollout status deployment/orders-$IDLE -n $NS --
```

```
timeout=180s
                    '''
               }
          }
          stage('Smoketest') {
               steps{sh'curl -fsS
http://orders-$IDLE.prod.svc.cluster.local:8080/actuator/health' }
          }
          stage('Approveswitch') {
               steps{input message: "Switch traffic to ${env.IDLE}?" }
          }
          stage('Switchtraffic') {
               steps {
                    sh '''
                       kubectl patch svc orders-svc -n $NS --type merge \
                         -p '{"spec":{"selector":
{"app":"orders","version":"'$IDLE'"}}}'
                    '''
               }
          }
     }
}
```

The smoke test uses a separate Service, `orders-<color>`, that always points to one color. In a GitOps setup, the same switch is done by changing the selector in Git, or by using **Argo Rollouts**, which supports blue-green natively.

> **Interview tip:** Mention the database challenge. A senior interviewer will ask, "What about schema changes?" Answer: backward-compatible migrations, so both versions work with the same database.

29. How do you automate a manual process using a Jenkins pipeline?

Short answer: First I write down the manual steps. Then I turn each step into a script or command, put them in a Jenkinsfile as stages, move secrets into Jenkins credentials, add parameters for the values that change, and choose a trigger: webhook, schedule or manual. Finally I add notifications and error handling.

My approach

1. **Document the manual process** — every command and every click.
2. **Script it** — shell, Python, Ansible or Terraform, kept in Git.
3. **Create stages** — one stage per logical step, so failures are easy to find.
4. **Parameterize** — environment, version, service name `as parameters`.

5. **Secure it** — credentials from the Jenkins store, never typed in scripts.
6. **Trigger** — webhook for code events, `cron` for scheduled tasks, manual for ops tasks.
7. **Handle errors** — `timeout`, `retry` for flaky network steps, `post` blocks for cleanup and alerts.
8. **Restrict access** — only the right team can run it (RBAC).

Hands-on experience: In my project, creating a new QA environment was a manual task that took half a day. I automated it with a parameterized pipeline: it ran Terraform to create the namespace resources and RDS database, applied the Helm chart, ran smoke tests, and posted the URL to Slack. It took 15 minutes, and anyone in the QA team could run it.

Example

```
pipeline {
    agent { label 'terraform' }
    parameters {
        choice(name:'ENV',choices: ['qa1', 'qa2', 'qa3'], description:
'Environment to create')
        booleanParam(name:'DESTROY', defaultValue: false, description:
'Destroy instead of create')
    }
    options{timeout(time:30,unit: 'MINUTES') }
    stages {
        stage('Terraform Init') {
            steps{dir("envs/${params.ENV}") { sh 'terraform init -
input=false' } }
        }
        stage('TerraformApply/Destroy') {
            steps {
                dir("envs/${params.ENV}") {
                    script {
                        defaction = params.DESTROY ? 'destroy' : 'apply'
                        sh"terraform ${action} -auto-approve -input=false"
                    }
                }
            }
        }
    }
    post {
        success{echo"Environment ${params.ENV} is ready" }
        failure{echo"Environment ${params.ENV} failed — check the logs" }
    }
}
```

> **Interview tip:** Give a real "before and after" example with time saved. Automation answers are judged by business impact.

30. How do you integrate tools like SonarQube, Maven, Docker, or Ansible into a Jenkins pipeline?

Short answer: There are two common ways. Some tools are configured in **Manage Jenkins → Tools** and used with the `tools` block, like Maven and JDK. Others are installed on the agent or come from a Docker image, and we call them with `sh`. Tools that need a server, like SonarQube, also need a plugin, server config and credentials.

How each tool is integrated

Tool	Integration
Maven / JDK	Manage Jenkins → Tools; `tools { maven 'maven-3.9'; jdk 'jdk17' }`
SonarQube	SonarQube Scanner plugin + server config + token; `withSonarQubeEnv` and `waitForQualityGate` (see Q18)
Docker	Docker CLI on the agent, or a Docker agent image; registry login with credentials (see Q61)
Ansible	Ansible plugin (`ansiblePlaybook` step) or `sh 'ansible-playbook ...'`; SSH key credential (see Q37)
Trivy	Installed on the agent or run as a container: `trivy image ...`
Kubernetes	`kubectl`/Helm on the agent + kubeconfig credential, or ArgoCD via GitOps

Modern preferred approach: run tool steps inside container images (`agent { docker { image 'maven:3.9-eclipse-temurin-17' } }` or Kubernetes pod templates). Then every build uses the exact same tool version, and agents don't need manual installs.

Example

```
pipeline {
    agent{label 'linux-docker' }
    tools {
        maven 'maven-3.9'
        jdk 'jdk17'
    }
    stages {
        stage('Build') { steps { sh 'mvn -B clean package' } }
        stage('Sonar') {
            steps { withSonarQubeEnv('sonarqube') { sh 'mvn -B sonar:sonar'}
}
        }
        stage('Docker') { steps { sh 'docker build -t orders:$BUILD_NUMBER .'
} }
    }
}
```

The names `maven-3.9` and `jdk17` must match the names configured in Manage Jenkins → Tools.

> **Interview tip:** Mention that you moved toward container-based tools for consistency. It shows you know the modern approach.

31. How do you pass parameters or variables to a Jenkins pipeline?

Short answer: For user input, we use the `parameters` block: string, choice, boolean, and so on. The job then shows **Build with Parameters**, and we read values `with params.NAME`. For variables, we use the `environment` block, `withEnv`, or set `env.VAR` inside a `script` block. Secrets come from credentials, not parameters.

Explanation

- **Parameter types:** `string`, `text`, `choice`, `booleanParam`, `password` (better to use credentials instead).
- **Read them:** `params.ENV` (always the right type) — also available as environment variables.
- **Environment variables:** pipeline-level or stage-level `environment { }`.
- **Dynamic values:** `env.IMAGE_TAG = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()`.
- **Pass to another job:** `build job: 'deploy-job', parameters: [string(name: 'IMAGE_TAG', value: env.IMAGE_TAG)]`.

- **Note:** parameters defined in the Jenkinsfile appear after the first run, because Jenkins reads them from the file.

Hands-on experience: Our deploy job had parameters for ENVIRONMENT (choice: dev, qa, prod), IMAGE_TAG (string), and RUN_SMOKE_TESTS (boolean). The CI pipeline called this job automatically with the new tag for Dev; for production, the release manager ran it manually.

Example

```
pipeline {
    agent any
    parameters {
        choice(name:'ENVIRONMENT',choices: ['dev', 'qa', 'prod'],
description: 'Target environment')
        string(name:'IMAGE_TAG',defaultValue: 'latest', description: 'Image
tag to deploy')
        booleanParam(name:'RUN_SMOKE_TESTS', defaultValue: true,
description: 'Run smoke tests')
    }
    environment {
        NAMESPACE = "orders-${params.ENVIRONMENT}"
    }
    stages {
        stage('Deploy') {
            steps {
                sh'helmupgrade--install orders ./chart -n $NAMESPACE --set
image.tag=$IMAGE_TAG'
            }
        }
        stage('Smoke Tests') {
            when{expression{params.RUN_SMOKE_TESTS } }
            steps{sh'./smoke-test.sh $NAMESPACE' }
        }
    }
}
```

> **Interview tip:** Mention the "first run registers parameters" behaviour and "never pass secrets as parameters." Both are practical points.

32. What is a Jenkins job?

Short answer: A Jenkins job (also called a project or item) is a configured task that Jenkins runs: for example, "build and deploy the orders service." Each time the job runs, it creates a **build** with its own number, console log and result. So the job is the definition,

and the build is one execution of it.

Explanation

- **Job types:** Freestyle, Pipeline, Multibranch Pipeline, Organization Folder.
- **A job contains:** source code settings, triggers, parameters, the pipeline script or Jenkinsfile path, and post-build actions.
- **A build contains:** build number, status (Success, Unstable, Failure, Aborted), console log, artifacts, test results.
- On disk: `$JENKINS_HOME/jobs/<job-name>/config.xml` holds the job config; `builds/` holds the build history.
- **Folders** group jobs by team or product, and can have their own permissions and credentials.

Hands-on experience: In my project, we organized jobs into folders by team: `payments/`, `orders/`, `platform/`. Each folder contained Multibranch jobs for that team's services, plus a few ops jobs like rollback and environment creation.

> **Interview tip:** Clearly separate job vs build vs pipeline vs stage. Interviewers use this question to check your basic vocabulary.

33. What common Jenkins issues have you faced (including in production), and how did you resolve them?

(Merged with source Q52: "What common Jenkins production issues have you handled?")

Short answer: The common ones I handled were: disk full on the controller or agents, agents going offline, credential errors, plugin compatibility problems after upgrades, webhook triggers not working, slow pipelines, and SonarQube Quality Gate timeouts. For each, I fixed the root cause and added prevention like monitoring, cleanup and testing upgrades in staging.

Issues and fixes

Issue	Root cause	Fix and prevention
`No space left on device`	Old builds, workspaces, Docker images	Cleanup, `buildDiscarder`, `cleanWs()`, scheduled `docker system prune`, disk alerts
Agent offline	Agent JVM killed (memory), network, Java mismatch	More memory, restart agent service, align Java versions, monitor node status

Issue	Root cause	Fix and prevention
Jenkins very slow / UI hangs	Builds running on controller, low heap, too many plugins	0 executors on controller, tune `-Xmx`, remove unused plugins
Plugin errors after upgrade	Incompatible plugin versions	Test upgrades on staging Jenkins, back up first, upgrade plugins in batches
Webhook not triggering	Wrong URL, firewall, missing trailing slash	Check GitHub "Recent Deliveries", fix network rules
Credential not found	Wrong ID or wrong folder scope	Fix ID, check credential scope
Quality Gate stuck	SonarQube webhook missing or wrong URL	Fix webhook, add `timeout`
Docker permission denied	Jenkins user not in `docker` group	`usermod -aG docker jenkins` + restart agent (or rootless/Kaniko builds)
GitHub API rate limit	Many branch scans with a personal token	Use a GitHub App credential
Builds stuck in queue	No agent with the required label, or executors full	Fix labels, add agents, use Kubernetes autoscaling agents

Hands-on experience (production story): After a plugin upgrade, all pipelines failed with a class-loading error. I rolled back the plugin from the backup copy in `plugins/`, restarted Jenkins, and pipelines recovered in about 20 minutes. After that, we built a staging Jenkins with the same plugins and always tested upgrades there first.

> **Interview tip:** Pick 2–3 issues and tell them as short stories: problem → how you found it → fix → prevention. Don't just read a list.

34. Where is Jenkins configuration stored when Jenkins is installed on an EC2 instance?

Short answer: Everything is stored in `JENKINS_HOME`, which is `/var/lib/jenkins` by default for Linux package installs. It holds the main `config.xml`, job configs, build history, plugins, credentials and secret keys. Service-level settings like Java options and port are in the systemd service configuration.

Inside `/var/lib/jenkins`

Path	Contains
`config.xml`	Main Jenkins system configuration
`jobs/`	Each job's `config.xml` and build history
`plugins/`	Installed plugins (`.jpi` files)
`credentials.xml`	Global credentials (encrypted)
`secrets/`	Encryption keys — required to decrypt credentials
`users/`	User accounts (for the Jenkins internal user database)
`nodes/`	Agent configurations
`workspace/`	Workspaces of builds that ran on the controller
`*.xml` (others)	Plugin configs, e.g. SonarQube, Git

Service settings: on systemd-based installs, override them with `sudo systemctl edit jenkins` (for example `Environment="JAVA_OPTS=-Xmx4g"` or `Environment="JENKINS_PORT=8080"`). Older installs used `/etc/default/jenkins` (Debian/Ubuntu) or `/etc/sysconfig/jenkins` (RHEL).

Hands-on experience: On EC2, I put `JENKINS_HOME` on a separate EBS volume and took daily EBS snapshots with AWS Backup. When we moved Jenkins to a bigger instance, I just detached the volume and attached it to the new instance. We also managed the system configuration as code with the **Configuration as Code (JCasC)** plugin, so a fresh Jenkins could be rebuilt from YAML.

> **Interview tip:** Always mention the `secrets/` folder. Without it, a restored Jenkins can't decrypt any credentials.

35. How do you back up Jenkins before upgrading it?

Short answer: I take a full backup of `JENKINS_HOME`, especially `config.xml`, `jobs/`, `plugins/`, `credentials.xml` and `secrets/`. On EC2, I also take an EBS snapshot. I keep the old `jenkins.war` and plugin versions so I can roll back. Then I test the upgrade on a staging Jenkins before production.

Step-by-step

1. **Announce a maintenance window.** Use **Prepare for Shutdown** so running builds finish and no new builds start.

2. **Stop Jenkins** for a consistent copy: `sudo systemctl stop jenkins`.
3. **Back up `JENKINS_HOME`**, excluding large disposable folders like workspaces:

```
sudo tar --exclude='workspace' --exclude='caches' \
  -czf /backup/jenkins-home-$(date +%F).tar.gz -C /var/lib jenkins
```

4. **EBS snapshot** of the Jenkins volume (fastest full restore).
5. **Save the current version info:** the WAR file (for package installs, `/usr/share/java/jenkins.war`) and the plugin list from Manage Jenkins → Plugins, or via the script console.
6. **Copy the backup off the server**, for example to S3: `aws s3 cp /backup/... s3://jenkins-backups/ .`
7. **Upgrade on staging first**, check plugin compatibility, run sample pipelines.
8. **Upgrade production**, start Jenkins, check logs and run a test job.
9. **Rollback plan:** restore the old WAR and `plugins/` folder, or restore from the snapshot.

For regular automatic backups, the **ThinBackup** plugin or a scheduled job that copies `JENKINS_HOME` to S3 works well.

Hands-on experience: Before each quarterly LTS upgrade, I took an EBS snapshot and a tar backup to S3, then upgraded our staging Jenkins first. Once, a plugin was not compatible with the new core version. We found it in staging, waited for the plugin fix, and production had zero issues.

> **Interview tip:** Mention LTS versions, staging first, and a tested restore. A backup you never tested is not a real backup.

36. How do you secure a Jenkins project (and Jenkins overall)?

Short answer: I secure Jenkins at several levels: login through SSO or LDAP, role-based access so each team sees only its own jobs, credentials stored in folder scope, HTTPS, no builds on the controller, and regular updates of Jenkins and plugins. I also limit who can change Jenkinsfiles that deploy to production.

Security checklist

- **Authentication:** SSO (SAML/OIDC) or LDAP/Active Directory. No anonymous access.
- **Authorization:** Role-based Authorization Strategy or Matrix-based security. Project-based permissions per folder.
- **Folders:** one folder per team, with folder-level credentials and permissions.
- **Credentials:** never in code; masked in logs; least-privilege tokens; IAM roles instead of keys.

- **Controller safety:** 0 executors on the built-in node; agent-to-controller access control stays on.
- **HTTPS** through a reverse proxy or load balancer; Jenkins not exposed directly to the internet.
- **CSRF protection** stays enabled (it is on by default).
- **Script security:** review Script Approval requests carefully; restrict the Script Console to admins.
- **Updates:** apply security advisories for Jenkins core and plugins quickly; remove unused plugins.
- **Audit:** Audit Trail plugin to log who changed what.
- **Production deploys:** only from `main`, with approval, and only on a dedicated deploy agent.

Hands-on experience: In my project, we integrated Jenkins with Azure AD SSO and used the Role-based Authorization Strategy plugin. Developers could build and view logs in their team folder, only DevOps leads could configure jobs, and only the release group could approve production deployments. Production kubeconfig credentials existed only in the `prod-deploy` folder.

> **Interview tip:** Organize your answer as authentication → authorization → credentials → infrastructure → updates. A structured answer sounds senior.

37. How do you integrate Jenkins with Ansible?

Short answer: We install Ansible on the Jenkins agent, store the SSH private key (and Ansible Vault password, if used) as Jenkins credentials, and run the playbook from the pipeline. We can use the Ansible plugin's `ansiblePlaybook` step or simply run `ansible-playbook` with `sh`. The playbooks and inventory live in Git.

Explanation

- **Agent:** Ansible installed (or an image with Ansible), with network access to the target servers.
- **Credentials:** SSH key as "SSH Username with private key"; Vault password as "Secret text" or "Secret file".
- **Inventory:** static file per environment, or dynamic inventory for AWS (`amazon.aws.aws_ec2`).
- **Typical use:** configure servers, deploy to VMs, patch OS, install agents — things outside Kubernetes.

Hands-on experience: In my project, some legacy Java apps still ran on EC2 VMs. The Jenkins pipeline built the JAR, uploaded it to Nexus, and then ran an Ansible playbook that downloaded the JAR on each server, updated the systemd service, and restarted it one server at a time (`serial: 1`), so there was no downtime.

Example

```
stage('DeploywithAnsible') {
    steps {
        withCredentials([
            sshUserPrivateKey(credentialsId: 'ansible-ssh-key',
keyFileVariable:'SSH_KEY', usernameVariable: 'SSH_USER'),
            file(credentialsId: 'ansible-vault-pass', variable:
'VAULT_PASS_FILE')
        ]){
            sh '''
              ansible-playbook -i inventories/prod/hosts.ini deploy.yml \
                --private-key "$SSH_KEY" -u "$SSH_USER" \
                --vault-password-file "$VAULT_PASS_FILE" \
                -e app_version=$BUILD_NUMBER
            '''
        }
    }
}
```

- `sshUserPrivateKey` writes the key to a temporary file and deletes it after the block.
- `--vault-password-file` decrypts secrets stored in Ansible Vault.
- `-e app_version=...` passes the build version to the playbook.

> **Interview tip:** Mention rolling deployment (`serial`) and Ansible Vault. These show you used Ansible for real deployments, not just a demo.

Part 5 — Git Triggers, Branching & End-to-End CI/CD

38. What is the difference between Continuous Delivery and Continuous Deployment?

Short answer: In both, every change is built, tested and made ready for release automatically. In **Continuous Delivery**, the final production release needs a manual approval. In **Continuous Deployment**, every change that passes all checks goes to production automatically, with no human step.

Explanation

	Continuous Integration	Continuous Delivery	Continuous Deployment
What is automated	Build + test on every commit	Everything up to production	Everything, including production
Production release	—	Manual approval (one click)	Automatic
Needs	Good tests	Good tests + reliable pipeline	Very strong tests, monitoring, fast rollback, feature flags

Hands-on experience: In my project, Dev and QA were Continuous Deployment: every merge to `main` went to Dev automatically and to QA after tests passed. Production was Continuous Delivery: the release manager approved the deployment using an `input` step (or a PR in the GitOps repo). We chose this because the business needed release control for production.

> **Interview tip:** Say which one your company used *and why*. Most companies use delivery for production and deployment for lower environments.

39. How do you trigger a pipeline only for a specific branch?

Short answer: In a Multibranch pipeline, all branches can run CI, and I use `when { branch 'main' }` so deploy stages run only on `main`. If the job itself should build only one branch, I set the branch filter in the job: "Branches to build" in a Pipeline job, or a name filter in the Multibranch branch source.

Options

1. **`when` condition (most common)** — the job runs for every branch, but certain stages only run for one branch.
2. **Pipeline job → Branches to build** — set `*/main`. The webhook triggers the job, but Jenkins builds only if `main` changed.
3. **Multibranch → Filter by name (with wildcards or regex)** — include only `main` `develop` `release/*`.
4. **Generic Webhook Trigger plugin** — read the `ref` from the webhook payload and use a regex filter, for example `^refs/heads/main$`.

Example

```
stage('Deploy to Dev') {
      when { branch 'main' }
      steps {
            sh './deploy.sh dev'
      }
}
stage('Deploy to Staging') {
      when { branch pattern: 'release/.*', comparator: 'REGEXP' }
      steps {
            sh './deploy.sh staging'
      }
}
```

`when { branch ... }` works in Multibranch pipelines, where `BRANCH_NAME` is set.

Hands-on experience: We used one Multibranch job per service. Feature branches ran build, tests and scans. Only `main` ran "Deploy to Dev", and only `release/*` branches deployed to staging.

> **Interview tip:** Explain the difference between "don't build this branch at all" (filters) and "build it but don't deploy it" (`when`).

40. How do you ignore pushes from certain branches?

Short answer: In a Multibranch pipeline, I exclude those branches in the branch source using **Filter by name** — for example, exclude `experimental/*` or `docs/*`. In a normal Pipeline job, I use a regex in "Branches to build". Inside the Jenkinsfile, I can also skip stages with `when { not { branch ... } }`.

Options

- **Multibranch → Filter by name (with wildcards):** Include `*`, Exclude `experimental/* docs/*`. Jenkins won't even create jobs for those branches.
- **Pipeline job → Branches to build (regex):** `:^(?!.*experimental).*$` — the leading `:` tells the Git plugin it is a regex.
- **Inside the Jenkinsfile:**

```
stage('Build') {
      when { not { branch 'experimental/*' } }
      steps { sh 'mvn clean package' }
}
```

- **Ignore by commit message or path:** the Git plugin's "Polling ignores commits with certain messages" or "in certain paths" (works for polling and the GitHub hook trigger), or `when { changeset }` to skip when only docs changed.

Hands-on experience: Our developers created many `spike/*` branches for experiments. Each push was using agents and filling the build queue. I excluded `spike/*` in the Multibranch branch source, which cut our build load by about 20%.

> **Interview tip:** Mention that filtering at the branch source level saves agent resources, not just pipeline time.

41. How do you trigger a pipeline when a pull request is raised?

Short answer: We use a Multibranch pipeline with the **GitHub Branch Source** plugin and enable "Discover pull requests". The GitHub webhook sends PR events, Jenkins creates a job for `each PR` (like PR-42), runs the pipeline, and reports the result back to GitHub as a status check. With branch protection, the PR can't be merged until Jenkins passes.

Steps

1. Multibranch job → GitHub branch source → Behaviours → **Discover pull requests from origin** (and from forks with a trust setting, for open-source style repos).
2. Choose the strategy: build the PR **merged with the target branch** (recommended) or the PR head only.
3. GitHub webhook includes **Pull requests** events.
4. In GitHub: **Branch protection rule** `on main` → require the Jenkins status check to pass.
5. In the Jenkinsfile, use `CHANGE_ID`, `CHANGE_TARGET` and `when { changeRequest() }` for PR-only logic.

Example

```
stage('PR Checks') {
    when { changeRequest() }
    steps {
        echo "Validating PR #${env.CHANGE_ID} into ${env.CHANGE_TARGET}"
        sh 'mvn verify'
    }
}
stage('Deploy') {
    when {
        allOf {
            branch 'main'
            not { changeRequest() }
        }
    }
    steps { sh './deploy.sh dev' }
}
```

Hands-on experience: In my project, every PR ran build, unit tests, SonarQube (with PR decoration in SonarQube editions that support it) and a Trivy filesystem scan. GitHub showed a green or red check on the PR, and `main` was protected, so no one could merge failing code.

> **Interview tip:** Mention "build the PR merged with the target branch". It catches integration problems before merge — interviewers like that detail.

42. How do you use Multibranch pipelines in your project?

Short answer: Every microservice repo has one Multibranch job. Jenkins discovers all branches and PRs automatically. Feature branches and PRs run CI only, `main` deploys to Dev, and release branches or tags go to staging and production with approval. All of this is controlled by one Jenkinsfile using `when` conditions.

How it works in practice

Branch type	What runs
`feature/*`	Build, unit tests, SonarQube, Trivy fs scan
`PR-*`	Same as feature + status check on GitHub
`main`	Full CI + Docker image + push + deploy to Dev
`release/*` or tags	Deploy to staging, then production after approval

Benefits we saw

- No manual job creation for new branches — Jenkins creates and deletes jobs automatically.
- Developers get feedback on their branch before raising a PR.
- One Jenkinsfile, one standard, for all branches.
- Old branch jobs are removed by the orphaned item strategy, keeping Jenkins clean.

Hands-on experience: We combined Multibranch with an **Organization Folder** for our GitHub organization. Any new repo with a Jenkinsfile got a Multibranch job automatically, so onboarding a new microservice to CI/CD took minutes instead of a day.

> **Interview tip:** Describe the branch-to-environment mapping. It is the main thing interviewers want to hear in this question.

43. How do you integrate Amazon S3 with Jenkins?

Short answer: We give Jenkins AWS access, ideally through an IAM role on the agent, and then use the AWS CLI (`aws s3 cp` / `aws s3 sync`) or the **Pipeline: AWS Steps** plugin (`s3Upload`, `s3Download`). We use S3 for build artifacts, test reports, static website deployments and Jenkins backups.

Explanation

- **Access:** IAM instance profile on EC2 agents, or IRSA for Kubernetes pod agents. If keys are needed, store them as AWS credentials in Jenkins and use `withAWS`.
- **Permissions:** least privilege — only the bucket and actions needed (`s3:PutObject`, `s3:GetObject`, `s3:ListBucket`).
- **Common uses:** store JARs or reports, deploy a React/Angular static site, store Jenkins backups, share files between pipelines.

Hands-on experience: For our frontend, the pipeline ran `npm ci` and `npm run build`, then synced the `build/` folder to an S3 bucket behind CloudFront and invalidated the cache. We also used S3 for nightly Jenkins backups with a lifecycle rule that moved old backups to Glacier.

Example

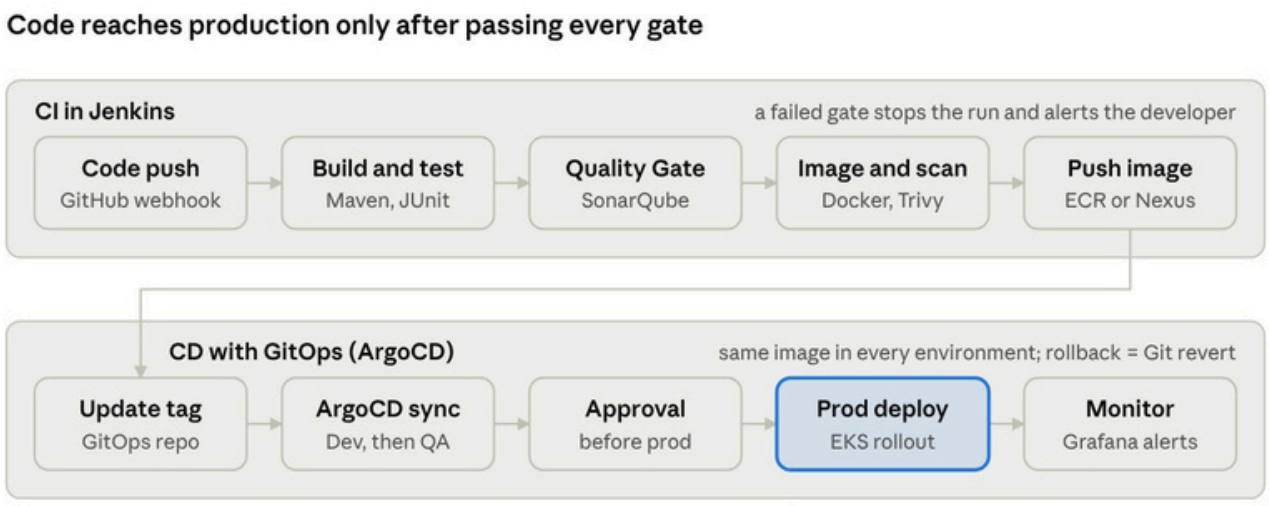
```
stage('DeployFrontend to S3') {
    steps {
        withAWS(region: 'ap-south-1', credentials: 'aws-deploy-creds') {
            sh '''
              aws s3 sync build/ s3://my-frontend-bucket/ --delete
              aws cloudfront create-invalidation --distribution-id
 E1234567890ABC--paths "/*"
            '''
        }
    }
}
```

- `withAWS` comes from the Pipeline: AWS Steps plugin. If the agent has an IAM role, you can skip `credentials` and just run the AWS CLI.
- `--delete` removes files from the bucket that no longer exist in the build.

> **Interview tip:** Say "IAM role, not access keys" first. Then give one concrete use case.

44. Explain the end-to-end CI/CD workflow using Jenkins.

Short answer: A developer pushes code to GitHub. A webhook triggers Jenkins. Jenkins builds and tests the code, checks quality in SonarQube, builds a Docker image, scans it with Trivy, and pushes it to the registry. Then Jenkins updates the image tag in the GitOps repo, ArgoCD deploys it to Kubernetes, and we monitor it. Production needs approval.



end-to-end CI/CD flow · 10 steps, CI in Jenkins, CD with ArgoCD

Any failed gate stops the run in CI; in CD the same image moves from Dev to QA to production, and rollback is a Git revert.

Step by step, the way I explain it in interviews

1. **Code** — developer creates a feature branch, pushes code, raises a PR.
2. **PR checks** — Jenkins runs build, tests and scans on the PR; GitHub shows the status.
3. **Merge** — after review and green checks, the PR is merged to `main`.
4. **CI on main** — webhook → Jenkins → checkout → Maven build → unit tests → SonarQube + Quality Gate.
5. **Package** — Docker build with tag = commit SHA → Trivy scan (fail on CRITICAL) → push to ECR/Nexus.
6. **CD to Dev** — Jenkins updates `envs/dev` in the GitOps repo → ArgoCD syncs to the Dev namespace.
7. **Test** — automated smoke and API tests in Dev, then promote to QA.
8. **Production** — approval (Jenkins `input` or a reviewed GitOps PR) → ArgoCD rolling or canary deployment to production.
9. **Monitor** — Prometheus/Grafana dashboards, alerts, logs; Slack notification of the release.
10. **Rollback if needed** — revert the GitOps commit; ArgoCD returns to the previous image.

Hands-on experience: In my project, this whole flow from merge to running in Dev took about 12 minutes. Production releases happened twice a week with approval, and the average rollback time was under 5 minutes because it was just a Git revert.

> **Interview tip:** Use real numbers: how long the pipeline takes, how often you release, how fast you roll back. Numbers make a workflow answer memorable.

45. How do you configure Jenkins from scratch?

Short answer: I launch a server (for example an EC2 instance), install Java and Jenkins, unlock it with the initial admin password, install the needed plugins, create admin users, and then set up security, agents, tools, credentials and integrations like GitHub, SonarQube and Slack. Finally, I set up backups, monitoring and Configuration as Code.
Step-by-step (Ubuntu on EC2)

1. **Server:** EC2 (for example t3.large), separate EBS volume for `/var/lib/jenkins`, security group allowing 8080 only from the load balancer or office IPs.
2. **Install Java and Jenkins (LTS):**

```
sudo apt update
sudo apt install -y fontconfig openjdk-17-jre
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
```

The Jenkins repository signing key is rotated from time to time, so always copy the current key URL from the official install page at jenkins.io.

3. Unlock: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`, open `http://<server>:8080`, paste it.
4. **Plugins:** suggested plugins, plus Pipeline, Git, GitHub Branch Source, Credentials Binding, Docker Pipeline, Kubernetes, SonarQube Scanner, Role-based Authorization Strategy, Configuration as Code.
5. **Admin and URL:** create the admin user, set the Jenkins URL.
6. **Security:** SSO or LDAP, role-based access, disable sign-up, HTTPS via ALB or Nginx.
7. **Controller executors = 0**, then add agents: SSH agents, the EC2 plugin, or Kubernetes pod agents.
8. **Tools:** JDK, Maven, SonarQube Scanner in Manage Jenkins → Tools.
9. **Credentials:** GitHub App, registry, Sonar token, kubeconfig — in folder scope where possible.
10. **Integrations:** SonarQube server, GitHub webhooks, Slack notifications.
11. **Operations:** JCasC YAML in Git, backups to S3 / EBS snapshots, monitoring with the Prometheus metrics plugin, build discarders.

Hands-on experience: In my last setup, I wrote the whole Jenkins configuration as JCasC YAML: security realm, roles, agents, tools and credentials references. We could rebuild Jenkins on a new server in about 30 minutes, which we tested during a disaster-recovery drill.

> **Interview tip:** Don't stop at "install and unlock". The interviewer wants to hear security, agents, backups and Configuration as Code.

46. Which CI/CD tool did you use, and why?

Short answer: I used Jenkins for CI and ArgoCD for CD. We chose Jenkins because it is very flexible, has plugins for everything in our stack, runs inside our own AWS network, and the team already had experience with it. ArgoCD handled Kubernetes deployments using GitOps, which gave us easy rollback and a clear audit trail.

Why Jenkins

- **Flexibility** — any language, any tool, any workflow.
- **Plugins** — GitHub, Docker, Kubernetes, SonarQube, AWS, Slack.
- **Self-hosted** — builds run inside our VPC with access to private resources; code never leaves our network.
- **Shared Libraries** — one standard pipeline for 25+ services.
- **Cost** — open source; we pay only for infrastructure.

Honest trade-offs: Jenkins needs maintenance: upgrades, plugins, backups and security patches. Tools like GitHub Actions or GitLab CI are easier to start with and are managed for you. For a new small project with code in GitHub, GitHub Actions can be a good fit.
Why ArgoCD for CD: the desired state lives in Git, ArgoCD keeps the cluster in sync, detects drift, and rollback is just a Git revert.

> **Interview tip:** Show balanced judgment. Saying "Jenkins was right for us because..., but for a small new project I'd consider GitHub Actions" sounds more senior than praising one tool blindly.

Part 6 — Jenkins Architecture: Nodes & Agents

47. Why do we need nodes in Jenkins?

Short answer: Nodes let Jenkins spread builds across many machines. Without them, every build runs on the controller, which becomes slow and unsafe. With nodes, we can run many builds in parallel, use different operating systems and tools, keep the controller stable, and isolate sensitive work like production deployments.

Reasons

- **Scale** — many teams can build at the same time.
- **Different environments** — Linux for Java and Docker, Windows for .NET, macOS for iOS builds.
- **Right tools on the right machine** — GPU, large memory, specific JDK versions.

- **Controller protection** — the controller handles UI, scheduling and config only. Builds that use a lot of CPU or run untrusted code don't touch it.
- **Security and network isolation** — a deploy node inside the production VPC, with no other jobs allowed.
- **Cost** — cloud agents start on demand and terminate when idle.

Hands-on experience: In my project, the controller had 0 executors. We had two static EC2 agents for Docker builds and used the Kubernetes plugin for dynamic pod agents. During release days, the cluster autoscaler added nodes and Jenkins could run 30+ builds at once. On quiet days, almost nothing was running, which kept costs low.

> **Interview tip:** Mention "0 executors on the controller" and "dynamic agents for cost". Those two points show production experience.

48. What are upstream and downstream jobs in Jenkins?

Short answer: If Job A triggers Job B after it finishes, Job A is the **upstream** job and Job B is the **downstream** job. We use this to chain work, for example a build job that triggers a deploy job. In pipelines we usually trigger downstream jobs with the `build` step.

Explanation

- **Upstream** = the job that runs first and triggers the next one.
- **Downstream** = the job that is triggered.
- **Ways to link them:**
  - In the upstream pipeline: `build job: 'deploy-orders', parameters: [...]`.
  - In the downstream pipeline: `triggers { upstream(upstreamProjects: 'build-orders', threshold: hudson.model.Result.SUCCESS) }`.
  - In Freestyle jobs: "Build other projects" post-build action, or "Build after other projects are built" trigger.
- `wait: true` (default) makes the upstream wait for the downstream result; `wait: false` just fires it and continues.

Hands-on experience: In my project, the CI job for each service triggered a common `deploy-service` job, passing the service name and image tag. The deploy job was shared and locked down with stricter permissions. Later, we moved most deployments to GitOps with ArgoCD, so the downstream deploy job was used only for legacy VM apps.

Example

```
stage('TriggerDeploy') {
    when{branch'main' }
    steps {
        build job: 'platform/deploy-service',
            parameters: [
                string(name: 'SERVICE', value: 'orders-service'),
                string(name: 'IMAGE_TAG', value: env.BUILD_NUMBER)
            ],
            wait: true
    }
}
```

> **Interview tip:** Add that too many chained jobs become hard to trace. Today, one pipeline with stages, or GitOps, is usually cleaner.

49. What are post-build actions in Jenkins?

Short answer: Post-build actions are steps that run after the main build, like publishing test results, archiving artifacts, sending notifications, cleaning the workspace or triggering other jobs. In Freestyle jobs they are configured in the UI. In pipelines we use the `post` block with conditions like `always`, `success` and `failure`.

`post` conditions in Declarative pipelines

Condition	Runs when
`always`	Every time, whatever the result
`success`	Build succeeded
`failure`	Build failed
`unstable`	Build is unstable (for example, test failures published by `junit`)
`aborted`	Build was aborted
`changed`	Result is different from the previous build
`fixed`	Previous build failed, this one succeeded
`regression`	Previous build succeeded, this one failed or is unstable
`cleanup`	After all other post conditions — good for cleanup

Hands-on experience: In our pipelines, `always` published JUnit reports, `failure` sent a Slack alert with the build link to the team channel, `fixed` sent a "back to green" message, and `cleanup` deleted the workspace. This kept the team informed without flooding them with messages for every successful build.

Example

```
post {
    always {
        junit allowEmptyResults: true, testResults: 'target/surefire-
reports/*.xml'
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true,
allowEmptyArchive: true
    }
    failure {
        slackSend channel: '#orders-ci', color: 'danger',
                  message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}
(${env.BUILD_URL})"
    }
    fixed {
        slackSend channel: '#orders-ci', color: 'good',
                  message: "FIXED: ${env.JOB_NAME} is green again"
    }
    cleanup {
        cleanWs()
    }
}
```

`slackSend` needs the Slack Notification plugin; `cleanWs` needs the Workspace Cleanup plugin.

> **Interview tip:** Mention `fixed` and `regression` — less common conditions that show you went beyond the basics.

50. What is an agent in Jenkins, and how does it connect to the controller?

Short answer: An agent is a machine or container that runs the build work for the Jenkins controller. The controller sends the job to an agent, the agent runs the steps and streams the logs back. Agents connect over SSH (controller connects to the agent), as inbound agents (agent connects to the controller), or are created on demand by cloud plugins like EC2 or Kubernetes.

Explanation

- **Executor** — a slot on an agent that runs one build at a time. An agent with 2 executors runs 2 builds in parallel.
- **Connection types:**
  - **SSH agent** — controller logs in over SSH with a key credential and starts `agent.jar`. Common for Linux VMs.
  - **Inbound agent** — the agent starts `agent.jar` and connects to the controller over TCP port 50000 or WebSocket. Useful when the agent is behind a firewall.
  - **Cloud agents** — EC2 plugin launches instances; Kubernetes plugin creates a pod per build and deletes it after.
- **In the Jenkinsfile,** the `agent` directive picks which agent runs the work (see Q14 and Q15).
- Agents need Java installed (same major version family as supported by the controller) for permanent agents.

Hands-on experience: We used SSH agents for our static EC2 build servers and Kubernetes pod agents for most pipelines. Each Kubernetes pod had containers for Maven, Docker-in-Docker alternatives like Kaniko, and kubectl, and was deleted after the build, so every build started clean.

Example (Kubernetes pod agent)

```
pipeline {
    agent {
        kubernetes {
            yaml '''
            apiVersion: v1
            kind: Pod
            spec:
              containers:
              - name: maven
                image:   maven:3.9-eclipse-temurin-17
                command: ["sleep"]
                args: ["infinity"]
            '''
        }
    }
    stages {
        stage('Build') {
            steps {
                container('maven') {
                    sh 'mvn -B clean package'
                }
```

```
                }
            }
        }
    }
```

`container('maven')` runs the steps inside the Maven container of the pod. The `sleep infinity` keeps thecontainer alive so Jenkins can run commands in it.

> **Interview tip:** Explain the difference between SSH and inbound agents. It's a common follow-up question.

51. What is the difference between master and slave (controller and agent) in Jenkins?

Short answer: Jenkins now uses the terms **controller** and **agent** instead of master and slave. The controller is the brain: it serves the UI, stores configuration, manages plugins and credentials, and schedules builds. Agents are the workers: they run the actual build steps. In production, builds should run only on agents.

Comparison

	Controller (old: master)	Agent (old: slave)
Main role	Manage and schedule	Execute builds
Stores	Config, jobs, build history, plugins, credentials	Workspaces, tool caches
UI	Yes	No
Count	Usually one (per Jenkins instance)	Many
Runs builds	Should not (0 executors)	Yes
If it fails	Whole Jenkins is down	Only builds on that agent are affected

How they talk: the controller sends build instructions to the agent through the remoting channel (over SSH, TCP or WebSocket), and the agent sends logs and results back.
Hands-on experience: In my project, the controller ran on one EC2 instance with an EBS volume for `JENKINS_HOME`. All builds ran on agents. When an agent had issues, we just took it offline and replaced it, without any impact on Jenkins itself.

> **Interview tip:** Use the modern terms "controller" and "agent" and mention that they replaced "master/slave". It shows you're current.

52. If a slave (agent) node is not working, how will you troubleshoot it?

Short answer: First I check the node's status and its log in **Manage Jenkins → Nodes**, which usually shows the exact reason. Then I check the agent machine: is it running, is the agent process alive, does it have disk space, memory and the right Java version, and can it reach the controller on the network. I fix the cause and relaunch the agent.

My step-by-step approach

1. **Node page and log:** Manage Jenkins → Nodes → (node) → **Log**. Look for messages like connection refused, authentication failed, or "disk space is too low".
2. **Is the machine up?** Check in the AWS console, then `ping` or SSH to it.
3. **Disk space:** Jenkins marks an agent offline automatically when free disk or temp space goes below the threshold. Check with `df -h`.
4. **Memory:** `free -m`, `dmesg -T | grep -i oom` — the agent JVM may have been killed.
5. **Java version:** `java -version` on the agent must be supported by the current Jenkins version.
6. **Network:** SSH agents need port 22 from controller to agent. Inbound agents need the controller reachable on port 50000 (or WebSocket over 443). Check security groups and firewalls: `nc -zv jenkins.example.com 50000`.
7. **Credentials:** for SSH agents, check the SSH key credential, the user, and the host key verification setting.
8. **Agent process:** for inbound agents, check the service: `systemctl status jenkins-agent` and its logs.
9. **Relaunch:** click **Launch agent** or restart the agent service; for cloud agents, check the cloud template and quotas (for example EC2 instance limits).
10. **Prevent:** monitor node status, add disk cleanup, use ephemeral agents (Kubernetes/EC2) that are recreated automatically.

Hands-on experience: One agent kept going offline every afternoon. The node log said the disk space was below the threshold. Docker images and old workspaces were filling the disk. I cleaned it, added a nightly cleanup job (`docker system prune -af --filter "until=72h"`), and later moved those builds to Kubernetes pod agents, which start clean every time.

> **Interview tip:** Start with "the node log tells me the reason". It shows you look at evidence before guessing.

Part 7 — Pipeline Design & Artifacts

53. How do you create a multi-stage pipeline in Jenkins?

Short answer: In a Declarative Jenkinsfile, I add several stage blocks inside stages. They run one after another by default. I can control them with when conditions, run some in parallel, give each its own agent, and group related stages using nested stages. Then I create a Pipeline or Multibranch job that points to this Jenkinsfile.

Explanation

- **Sequential by default** — if one stage fails, the next stages don't run.
- `Conditional` — `when { branch 'main' }` or `when { expression { ... } }`.
- **Parallel** — independent stages side by side (Q16).
- **Nested (sequential) stages** — a parent stage containing its own stages block, useful to group stages that share an agent.
- `Per-stage settings` — `agent`, `environment`, `options { timeout(...) }`, `post`, `input`.

Example

```
pipeline {
    agent none
    options { timestamps() }
    stages {
        stage('CI') {
            agent { label 'maven-jdk17' }
            stages {
                stage('Build') { steps { sh 'mvn -B clean package -
DskipTests' } }
                stage('Test') { steps { sh 'mvn -B test' } }
            }
        }
        stage('Deploy to Dev') {
            when { branch 'main' }
            agent { label 'deploy' }
            steps { sh './deploy.sh dev' }
        }
        stage('Deploy to Prod') {
            when { branch 'main' }
            agent none
            steps {
managers'           input message: 'Deploy to production?', submitter: 'release-

                node('deploy-prod') {
```

```
                    sh './deploy.sh prod'
                }
            }
        }
    }
}
```

- The **CI** stage groups Build and Test on the same Maven agent, so they share the workspace.
- The **Prod** stage waits for approval without holding an agent, then grabs the `deploy-prod` agent only after approval. (A `node` step inside Declarative `steps` is allowed.)

Hands-on experience: Our pipelines had about 10 stages. Grouping CI stages under one agent saved time on checkout, and using `agent none` for approval stages stopped agents from sitting idle for hours while waiting for a human.

> **Interview tip:** Mention "don't hold an executor during `input`". It's a classic mistake that experienced interviewers look for.

54. Explain the Jenkins pipeline script you wrote in your project.

Short answer: I wrote a Declarative Jenkinsfile for our Java microservices. It checks out the code, builds and tests with Maven, runs SonarQube and waits for the Quality Gate, builds a Docker image tagged with the commit ID, scans it with Trivy, pushes it to ECR, and then updates the image tag in our GitOps repo so ArgoCD deploys it to Dev. It sends Slack notifications and cleans the workspace at the end.

The script

```
pipeline {
    agent{label'linux-docker' }

    options {
        timeout(time:45, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '30'))
        disableConcurrentBuilds()
        timestamps()
    }

    environment {
        AWS_REGION = 'ap-south-1'
        REGISTRY   ='123456789012.dkr.ecr.ap-south-1.amazonaws.com'
        IMAGE      ="${REGISTRY}/orders-service"
    }
```

```
    stages {
        stage('Checkout') {
            steps {
                checkout scm
                script {
                    env.TAG = sh(script: 'git rev-parse --short HEAD',
returnStdout: true).trim()
                }
            }
        }

        stage('Build & Unit Test') {
            steps { sh 'mvn -B clean verify' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }

        stage('SonarQube') {
            steps {
                withSonarQubeEnv('sonarqube') { sh 'mvn -B sonar:sonar' }
            }
        }

        stage('Quality Gate') {
            steps {
                timeout(time: 10, unit: 'MINUTES') {
                    waitForQualityGate abortPipeline: true
                }
            }
        }

        stage('Docker Build') {
            steps { sh 'docker build -t $IMAGE:$TAG .' }
        }

        stage('Trivy Scan') {
            steps {
                sh 'trivy image --exit-code 1 --severity CRITICAL --ignore-
unfixed $IMAGE:$TAG'
            }
        }

        stage('Push to ECR') {
            steps {
                sh '''
                  aws ecr get-login-password --region $AWS_REGION | \
                    docker login --username AWS --password-stdin $REGISTRY
```

```
                    docker push $IMAGE:$TAG
                '''
            }
        }

        stage('Update GitOps (Dev)') {
        when { branch 'main' }
        steps {
                withCredentials([usernamePassword(credentialsId: 'github-
gitops',
                                                  usernameVariable:
'GIT_USER',
                                                  passwordVariable:
'GIT_TOKEN')]) {
                    sh '''
                      rm -rf gitops
                      git clone
https://$GIT_USER:$GIT_TOKEN@github.com/acme/gitops.git gitops
                      cd gitops/apps/orders-service/envs/dev
                      yq -i '.image.tag = strenv(TAG)' values.yaml
                      git config user.email "jenkins@acme.com"
                      git config user.name "jenkins"
                      git commit -am "orders-service: deploy $TAG to dev"
                      git push origin main
                    '''
                }
            }
        }
    }

    post {
        success { slackSend channel: '#orders-ci', color: 'good',          message:
"SUCCESS ${env.JOB_NAME} #${env.BUILD_NUMBER} — tag ${env.TAG}" }
        failure { slackSend channel: '#orders-ci', color: 'danger', message:
"FAILED ${env.JOB_NAME} #${env.BUILD_NUMBER} — ${env.BUILD_URL}" }
        cleanup { cleanWs() }
    }
}
```

How I explain it in the interview

- **Agent and options** — runs on a Docker-capable agent, with a 45-minute timeout, build retention, no concurrent builds, and timestamps in logs.
- **Tag** — the short Git commit SHA, so every image can be traced to the exact code.
- **Build & test** — `mvn verify` compiles, runs tests and creates the JaCoCo report; `junit` publishes results.

- **SonarQube + Quality Gate** — the pipeline stops if quality rules fail.
- **Trivy** — fails on CRITICAL vulnerabilities that have a fix available.
- **Push** — the agent's IAM role is used for ECR login; no keys stored.
- **GitOps update** — only on `main`; Jenkins changes the tag with `yq` and pushes; ArgoCD does the deployment.
- **Post** — Slack messages and workspace cleanup.

In the real project this logic lived in our Shared Library, and each service's Jenkinsfile just called it with the service name.

> **Interview tip:** Practise explaining this script in 2 minutes, stage by stage, with the *reason* for each one. This is the most common deep-dive question in Jenkins interviews.

55. What stages do you include after the Maven build and vulnerability scanning?

Short answer: After the Maven build and code/dependency scanning, we build the Docker image, scan the image with Trivy, push it to the registry, and update the deployment manifest. Then we deploy to Dev, run smoke and integration tests, and promote to higher environments with approval. Notifications and cleanup run at the end.
Typical order after build + scan

1. **Quality Gate** (if not done yet) — stop if SonarQube fails.
2. **Publish artifact** — `mvn deploy` the JAR to Nexus (for library or VM-based apps).
3. **Docker build** — image tagged with commit SHA.
4. **Image scan** — Trivy for OS and library vulnerabilities in the final image.
5. **Push image** — to ECR/Nexus; optionally sign it (for example with Cosign).
6. **Update manifest** — Helm values or Kustomize tag in the GitOps repo.
7. **Deploy to Dev** — ArgoCD sync or `helm upgrade`.
8. **Smoke tests** — health endpoint and key APIs.
9. **Integration / API tests** — Postman/Newman or REST-assured tests against Dev/QA.
10. **Approval** — before production.
11. **Production deploy** — rolling, blue-green or canary.
12. **Post** — notifications, reports, cleanup.

Hands-on experience: In my project, we had two scans: `trivy fs` on the source after the Maven build (to catch vulnerable dependencies early), and `trivy image` after the Docker build (to catch OS package issues in the base image). Then push to ECR, GitOps update, and smoke tests in Dev.

> **Interview tip:** Explain why you scan twice: source dependencies early, the final image later. It shows real security thinking.

56. How would you design a single pipeline to deploy 10 microservices to Dev?

Short answer: I would keep the logic generic and data-driven: one pipeline that reads a list of services, detects which ones changed, and builds and deploys only those, in parallel. The common steps come from a Shared Library. For the deploy, I would update each service's tag in the GitOps repo in one commit, and ArgoCD syncs all of them to Dev.
Design points

- **Service list** — kept in a config file (for example `services.yaml`) or a parameter, not hardcoded in many places.
- **Change detection** — build only services whose folder changed (`git diff --name-only`), with an option to force all.
- **Parallel builds** — dynamic `parallel` map, with a limit so agents aren't overloaded.
- **Kubernetes pod agents** — so 10 builds can run at the same time.
- **Fail behaviour** — one failing service should not hide others; report each result.
- **Deploy** — one GitOps commit for all changed services, or ArgoCD **ApplicationSet** to manage all 10 apps.
- **Parameters** — `SERVICES` (multi-select or comma list) and `FORCE_ALL`.

Example (Declarative with a `script` block)

```
pipeline {
    agent { label 'linux-docker' }
    parameters {
        booleanParam(name: 'FORCE_ALL', defaultValue: false, description:
'Build all services')
    }
    stages {
        stage('Detect changes') {
            steps {
                script {
                    def all = ['orders', 'payments', 'cart', 'users',
'catalog',
                               'inventory', 'shipping', 'notify', 'search',
'gateway']
                    if (params.FORCE_ALL) {
                        env.CHANGED = all.join(',')
                    } else {
                        def diff = sh(script: 'git diff --name-only HEAD~1
```

```
HEAD', returnStdout: true).trim()
                        env.CHANGED=all.findAll { svc ->
diff.contains("services/${svc}/") }.join(',')
                    }
                    echo"Servicestobuild: ${env.CHANGED}"
                }
            }
        }
        stage('Build & Push') {
            when{expression{env.CHANGED?.trim() } }
            steps {
                script {
                    def branches = [:]
                    env.CHANGED.split(',').each { svc ->
                        branches[svc] = {
                            node('linux-docker') {
                                checkout              scm
                                dir("services/${svc}")      {
                                sh"docker build -t
myrepo/${svc}:${env.BUILD_NUMBER} ."
                                    sh"docker push
myrepo/${svc}:${env.BUILD_NUMBER}"
                                }
                            }
                        }
                    }
                    parallel branches
                }
            }
        }
        stage('Deploy to Dev (GitOps)') {
            when{expression{env.CHANGED?.trim() } }
            steps {
                sh'./scripts/update-gitops-tags.sh dev "$CHANGED"
"$BUILD_NUMBER"'
            }
        }
    }
}
```

- `diff.contains("services/${svc}/")` builds only services whose folder changed.
- `parallel branches` runs all builds at the same time, each on its own agent.
- The script updates all tags in one GitOps commit; ArgoCD deploys them together.
- `HEAD~1` is a simplification; in Multibranch builds, comparing against the last successful build's commit (`GIT_PREVIOUS_SUCCESSFUL_COMMIT`) is more reliable.

Hands-on experience: In our monorepo, this approach cut the average build from 35 minutes (all services, one by one) to about 8 minutes (only changed services, in parallel).

> **Interview tip:** Use the words "data-driven", "build only what changed" and "parallel". Also say you'd still prefer separate pipelines per service if the repos are separate.

57. How do you make sure the test stage runs only after a successful build?

Short answer: Stages in a Declarative pipeline run in order, and if the build stage fails, Jenkins stops and skips the next stages automatically. So I put Test after Build and make sure build commands return a non-zero exit code on failure. I avoid hiding errors with things like `|| true`. For unstable builds, I use `skipStagesAfterUnstable()` or a `when` condition.

Explanation

- **Default behaviour:** a failed `sh` step (non-zero exit code) fails the stage, and later stages are skipped.
- **Don't hide failures:** `sh 'mvn package || true'` or a wide `catchError` would let Test run after a failed build.
- **Unstable builds:** test failures published with `junit` mark the build UNSTABLE, not FAILED. By default later stages still run. To stop them, use `options { skipStagesAfterUnstable() }`.
- **Explicit check:** `when { expression { currentBuild.currentResult == 'SUCCESS' } }`.
- **Different agents:** if Test runs elsewhere, `stash` the build output and `unstash` it in Test — the unstash fails if the build didn't produce it.

Example

```
pipeline {
    agent any
    options{skipStagesAfterUnstable() }
    stages {
        stage('Build') {
            steps { sh 'mvn -B clean package -DskipTests' }
        }
        stage('Test') {
            steps { sh 'mvn -B test' }
            post { always { junit 'target/surefire-reports/*.xml' } }
        }
        stage('Deploy') {
            when { expression { currentBuild.currentResult == 'SUCCESS' } }
            steps { sh './deploy.sh dev' }
        }
    }
}
```

Hands-on experience: Once, a Deploy stage ran even though tests had failed, because the tests were published with `junit` and the build was only UNSTABLE. I added `skipStagesAfterUnstable()` and the `when` check, so deployments only happen when everything is green.

> **Interview tip:** The UNSTABLE vs FAILURE difference is the expert detail here. Most candidates miss it.

58. What is artifact management in CI/CD?

Short answer: Artifact management means storing the outputs of our builds — JAR files, Docker images, Helm charts, npm packages — in a central, versioned repository like Nexus, Artifactory or ECR. We build once, store the artifact with a unique version, and deploy that same artifact to every environment. This gives traceability, rollback and consistency.

Key ideas

- **Versioned and immutable** — each artifact has a unique version (for example `1.4.2` or the commit SHA) and is never overwritten.
- **Build once, deploy many** — the same artifact goes to Dev, QA and Prod.
- **Traceability** — version → commit → build number → test and scan results.
- **Easy rollback** — the previous version is still in the repository.
- **Retention policies** — clean up old snapshots to save storage.

- **Proxy repositories** — Nexus also caches Maven Central and Docker Hub, making builds faster and protecting against outages and rate limits.
- **Security** — access control, and scanning artifacts in the repository.

Hands-on experience: In my project, Nexus had three Maven repositories: `maven-releases`, `maven-snapshots`, and a `maven-central` proxy. Docker images went to ECR with tag immutability turned on. Jenkins only archived small reports; real artifacts went to Nexus/ECR, because storing big files in Jenkins fills the controller disk.

Example

`pom.xml` :

```
<distributionManagement>
  <repository>
    <id>nexus-releases</id>
    <url>https://nexus.example.com/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>https://nexus.example.com/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

Jenkins stage:

```
stage('Publish to Nexus') {
    steps {
        configFileProvider([configFile(fileId: 'maven-settings', variable:
'MAVEN_SETTINGS')]) {
            sh 'mvn -B -s $MAVEN_SETTINGS deploy -DskipTests'
        }
    }
}
```

- The `<id>` values must match server entries in `settings.xml`, which holds the Nexus credentials.
- `configFileProvider` (Config File Provider plugin) supplies the `settings.xml` securely from Jenkins.

> **Interview tip:** Say "build once, deploy many" and "immutable versions". Those are the two ideas interviewers listen for.

Part 8 — Jenkins + Docker & Kubernetes

59. How does Jenkins connect with Kubernetes?

Short answer: There are two connections. First, Jenkins can **use Kubernetes to run its agents** through the Kubernetes plugin: each build gets a temporary pod. Second, Jenkins can **deploy to Kubernetes**, either directly with kubectl or Helm using a kubeconfig or service account credential, or indirectly through GitOps, where Jenkins updates Git and ArgoCD deploys.

1. Kubernetes as the agent platform

- Install the **Kubernetes** plugin.
- Manage Jenkins → Clouds → **New cloud → Kubernetes**: cluster URL, namespace (for example `jenkins-agents`), credentials. If Jenkins runs inside the same cluster, it can use its own service account.
- Define **pod templates** (in the UI, JCasC or inline YAML in the Jenkinsfile).
- Each build creates a pod, runs the steps inside its containers, and deletes the pod at the end.
- The service account needs RBAC permission to create and delete pods in that namespace only.

2. Deploying to Kubernetes

Approach	How it works	Notes
`kubectl` / Helm from Jenkins	Kubeconfig as a Secret file credential, or an EKS IAM role + `aws eks update-kubeconfig`	Jenkins needs cluster access; use a limited service account per namespace
GitOps (preferred)	Jenkins updates the image tag in Git; ArgoCD pulls and syncs	Jenkins needs no cluster credentials; rollback = Git revert

Hands-on experience: In my project, Jenkins agents ran as pods on a separate EKS node group. For deployments, Jenkins had no access to the production cluster at all. It only pushed tag changes to the GitOps repo, and ArgoCD inside the cluster applied them. This removed a big security risk: production cluster credentials stored in Jenkins.

Example (direct deploy for a Dev cluster)

```
stage('DeploytoDev EKS') {
     steps {
          withCredentials([file(credentialsId: 'kubeconfig-dev', variable:
'KUBECONFIG')]) {
               sh '''
                  kubectl -n orders-dev apply -f k8s/
                  kubectl -n orders-dev rollout status deployment/orders-service
--timeout=180s
               '''
          }
     }
}
```

Binding the file to the `KUBECONFIG` variable makes `kubectl` use it automatically, and the temporary file is removed after the block.

> **Interview tip:** Mention both directions (agents *on* Kubernetes, deployments *to* Kubernetes) and say why GitOps is safer for production.

60. How do you change the image tag in Kubernetes using Jenkins?

Short answer: There are a few ways: `kubectl set image` for a quick direct update, `helm upgrade --set image.tag=...` if we use Helm, or — our preferred way — updating the tag in the GitOps repo with `yq` or `kustomize edit set image`, and letting ArgoCD deploy it. In all cases, the tag is a unique value like the commit SHA, not `latest`.
Options

1. **kubectl (direct):**

   ```
   kubectl -n orders-dev set image deployment/orders-service orders-
   service=$IMAGE:$TAG
   kubectl -n orders-dev rollout status deployment/orders-service
   ```

   `orders-service=` is the **container name** inside the Deployment, not the Deployment name.
2. **Helm:**

   ```
   helm upgrade --install orders ./charts/orders -n orders-dev --set
   image.tag=$TAG --wait
   ```

3. **Kustomize (GitOps repo):**

```
cd overlays/dev
kustomize edit set image myrepo/orders-service=myrepo/orders-service:$TAG
git commit -am "orders-service: $TAG to dev" && git push
```

4. **Helm values in the GitOps repo with yq:**

```
yq -i '.image.tag = strenv(TAG)' apps/orders/envs/dev/values.yaml
git commit -am "orders-service: $TAG to dev" && git push
```

Why GitOps is preferred: Git becomes the record of what is deployed, ArgoCD fixes manual drift, and rollback is just reverting a commit. With `kubectl set image`, the cluster and Git can drift apart.

Hands-on experience: We started with `kubectl set image`. Later, we found that someone had changed the image manually during an incident, and nobody knew which version was running. After moving to GitOps with `yq` updates, the repo always showed the truth, and ArgoCD flagged any manual change as "OutOfSync".

> **Interview tip:** Mention `rollout status` after a direct update. It makes the pipeline fail if the new pods don't become ready.

61. How does Jenkins integrate with Docker to build and push images?

Short answer: The Jenkins agent has Docker (or a builder like Kaniko or BuildKit). The pipeline builds the image from the Dockerfile, tags it with a unique version, logs in to the registry using a Jenkins credential or an IAM role, and pushes it. We can use plain `sh` Docker commands or the Docker Pipeline plugin's `docker.build` and `docker.withRegistry`.

Explanation

- **Agent needs Docker access** — the `jenkins` user in the `docker` group on a VM agent, or a Docker-in-Docker sidecar. On Kubernetes, many teams use **Kaniko** or **BuildKit** to avoid mounting the host Docker socket, which is a security risk.
- **Registry login** — Docker Hub/Nexus with a username/password credential; ECR with `aws ecr get-login-password` and an IAM role.
- **Tagging** — commit SHA or build number; optionally also a moving tag like `dev-latest`, but deploy by the immutable tag.
- **Clean up** — remove local images after push, or prune regularly, to avoid filling the agent disk.

Example (Docker Pipeline plugin)

```
stage('Build&Push Image') {
    steps {
        script {
            defimg = docker.build("mycompany/orders-
service:${env.GIT_COMMIT.take(7)}")
            docker.withRegistry('https://registry.hub.docker.com',
'dockerhub-creds') {
                img.push()
                img.push('dev-latest')
            }
        }
    }
}
```

Example (ECR with shell commands)

```
stage('Push to ECR') {
    steps {
        sh '''
          aws ecr get-login-password --region ap-south-1 | \
            docker login --username AWS --password-stdin
123456789012.dkr.ecr.ap-south-1.amazonaws.com
          docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-
service:$TAG
          docker rmi 123456789012.dkr.ecr.ap-south-1.amazonaws.com/orders-
service:$TAG || true
        '''
    }
}
```

`--password-stdin` keeps the password out of the process list and logs.

Hands-on experience: On VM agents, we used Docker directly. When we moved builds to Kubernetes pod agents, we switched to Kaniko so we didn't need privileged pods or the host Docker socket. Kaniko built and pushed the image to ECR using the pod's IRSA role.

> **Interview tip:** Say why mounting `/var/run/docker.sock` is risky (it gives root-level control of the node) and name Kaniko or BuildKit as the safer option.

62. What is the purpose of `withDockerRegistry` in Jenkins?

Short answer: `withDockerRegistry` is a step from the Docker Pipeline plugin. It logs in to a Docker registry using a Jenkins credential, runs the commands inside the block with that login, and then logs out. So we can `docker push` or `docker pull` from a private

registry without writing the password in the script.

Explanation

- It takes the registry URL and a **username/password credential ID**.
- An empty `url` means Docker Hub.
- Inside the block, any `docker` command (including `sh 'docker push ...'`) is authenticated.
- The credentials are kept in a temporary Docker config for the block and removed afterwards.
- `docker.withRegistry(url, credId) { }` is the scripted-style version of the same thing.
- For **ECR**, the Amazon ECR plugin lets you use a credential ID like `ecr:ap-south-1:aws-creds`, or you simply use aws `ecr get-login-password` with an IAM role.

Example

```
stage('PushImage') {
    steps {
        withDockerRegistry([credentialsId: 'nexus-docker-creds', url:
'https://nexus.example.com:8083']) {
            sh 'docker push nexus.example.com:8083/orders-service:$TAG'
        }
    }
}
```

Hands-on experience: We pushed images to a Nexus Docker hosted repository on port 8083. Using `withDockerRegistry` with a Jenkins credential meant the password never appeared in the Jenkinsfile or logs, and each team had its own credential scoped to its folder.

> **Interview tip:** The key point is "login only for the block, credentials from the Jenkins store, auto logout". That's what the interviewer is checking.

Part 9 — Troubleshooting & Production Scenarios

63. A Jenkins stage fails because of a credential issue. How do you fix it?

Short answer: First I read the exact error: "credentials not found", "authentication failed", "permission denied" or "token expired". Then I check that the credential ID in the Jenkinsfile matches the stored credential, that it is in the right scope or folder, that it is

the right type, and that the secret itself is still valid. I fix it, re-run the stage, and add expiry monitoring.

My step-by-step approach

1. **Read the error carefully:**
   - `CredentialNotFoundException` / "Could not find credentials matching ID" → wrong ID or wrong scope.
   - `401 Unauthorized` / "authentication failed" → wrong or expired password/token.
   - `403 Forbidden` / "access denied" → credential is valid but lacks permission.
   - SSH "Permission denied (publickey)" → wrong key or user, or key not on the target.
2. **Check the ID** — IDs are case-sensitive: `nexus-creds` is not `Nexus-Creds`.
3. **Check the scope** — a credential in folder `payments/` is not visible to jobs in `orders/`. System-scoped credentials are not available to pipelines.
4. **Check the type** — `usernamePassword` binding on a "Secret text" credential fails.
5. **Check validity** — GitHub tokens, registry passwords and cloud keys expire or get rotated. Test manually from the agent: `docker login`, `git ls-remote`, `aws sts get-caller-identity .`
6. **Check permissions on the target** — the Nexus user may lack deploy rights, or the IAM role may lack `ecr:PutImage`.
7. **Fix** — update the credential value (keep the same ID), or fix the ID/scope in the Jenkinsfile.
8. **Re-run** — use **Restart from Stage** to save time.
9. **Prevent** — track token expiry dates, use GitHub Apps and IAM roles (no expiring personal tokens), and alert before expiry.

Hands-on experience: One Monday, all pushes to Nexus failed with 401. The service account password had been rotated by the security team over the weekend, but the Jenkins credential wasn't updated. I updated the credential with the same ID, re-ran the failed builds, and we added Jenkins to the rotation checklist. Later we moved the password into AWS Secrets Manager, synced into Jenkins, so rotation updated it automatically.

> **Interview tip:** Map each error message to its cause (401 vs 403 vs "not found"). That precision impresses interviewers.

64. How do you require manual approval before a job or stage runs?

Short answer: In the pipeline, I add an `input` step before the production stage. The pipeline pauses and only users or groups listed in `submitter` can click Proceed or Abort. I wrap it in a `timeout` so it doesn't wait forever, and I run it without holding an agent. For pure manual jobs, I restrict "Build" permission to the release team using RBAC.

Explanation

- `input` shows a Proceed/Abort prompt in the build page and Stage View.
- `submitter` limits who can approve (user IDs or group names, comma-separated).
- `submitterParameter` records who approved, useful for audit.
- `parameters` inside `input` lets the approver choose options (for example a release note or a choice).
- Put `input` in a stage with `agent none`, or in the stage's `input` directive, so no executor is blocked.

Example

```
stage('Approve Production') {
    agent none
    steps {
        timeout(time: 4, unit: 'HOURS') {
            script {
                def approver = input message: "Deploy ${env.TAG} to
PRODUCTION?",
                                     ok: 'Deploy',
                                     submitter:    'release-managers',
                                     submitterParameter:
'APPROVER'
                echo "Approved by: ${approver}"
            }
        }
    }
}
stage('Deploy Production') {

    agent { label 'deploy-prod' }
    steps { sh './deploy.sh prod $TAG' }
}
```

- If nobody approves within 4 hours, the build is aborted.
- With only `submitterParameter` set, `input` returns the approver's ID.

Hands-on experience: Our production stage paused for approval from the `release-managers` group, and the approver's name was posted to Slack and the change ticket. For GitOps production releases, approval was a pull request in the GitOps repo with

CODEOWNERS review instead.

> **Interview tip:** Mention `submitter`, `timeout` and "no executor held while waiting". These three make the answer complete.

65. What happens if the Quality Gate fails and `abortPipeline: false` is set?

Short answer: The pipeline does **not** fail automatically. `waitForQualityGate` returns the result, and the pipeline continues to the next stages. It is then our job to check the status and decide — for example mark the build UNSTABLE, warn the team, or block only production deployments.

Explanation

- `abortPipeline: true` → a failed gate fails the build immediately.
- `abortPipeline: false` → the step returns an object with a status (`OK`, `ERROR`, and so on) and does nothing else.
- If we ignore the result, poor-quality code can be deployed. That's the risk.

When it is useful

- Gradual adoption on a legacy project: report without blocking at first.
- Non-blocking on feature branches, blocking on `main` and release branches.

Example

```
stage('Quality Gate') {
    steps {
        timeout(time:5,unit: 'MINUTES') {
            script {
                defqg=waitForQualityGate abortPipeline: false
                if(qg.status != 'OK') {
                    if(env.BRANCH_NAME == 'main') {
                        error "Quality Gate failed: ${qg.status}"
                    } else {
                        unstable "Quality Gate failed: ${qg.status} (non-
blocking on feature branches)"
                    }
                }
            }
        }
    }
}
```

Hands-on experience: When we first introduced SonarQube on an old service, it had hundreds of existing issues. We used `abortPipeline: false` and marked builds UNSTABLE for two sprints while the team cleaned up. Then we switched to `abortPipeline: true` with a "new code" gate.

> **Interview tip:** Say clearly: "With `false`, it's my responsibility to check the status." Then give the gradual-adoption use case.

66. If the Maven build stage is taking too long, what could be the reasons?

Short answer: The usual reasons are: dependencies downloading every build because there's no local cache or Nexus proxy, slow tests running one by one, `clean` rebuilding everything, a large multi-module project built without parallel threads, low CPU or memory on the agent, snapshot update checks, and network slowness to the repository. I check the build log timestamps to find which part is slow, then fix that part.

Reasons and fixes

Reason	How I confirm	Fix
Downloading dependencies every build	Many "Downloading from" lines	Persistent `~/.m2` cache, Nexus `proxy`, `dependency:go-offline` in Docker layers
Ephemeral agent with empty cache	Every Kubernetes pod starts clean	Mount a PVC or cache volume for .m2
Slow or many tests	Surefire report durations	Parallel tests, move integration tests to a nightly job
No parallel module build	Multi-module build runs serially	`mvn -T 1C`
Low agent resources	CPU at 100%, GC logs, swap	Bigger agent, set `MAVEN_OPTS=-Xmx2g` Avoid
Snapshot update checks	"Checking for updates" lines	`-U` on every build, set update policy
Slow network to Nexus/Maven Central	Slow download speeds	Nexus in the same region/VPC
Heavy plugins (e.g., enforcer, javadoc, shading)	Timestamps around those plugins	Skip non-essential plugins in CI profiles

Hands-on experience: Our Maven stage took 14 minutes on Kubernetes pod agents. The log showed about 8 minutes of dependency downloads, because each pod started with an empty `.m2`. I added a shared cache volume and pointed Maven to our Nexus proxy in the same VPC. The stage dropped to around 4 minutes.

> **Interview tip:** Start with "I look at timestamps in the log to see where time goes." Data first, then fixes.

67. If a pipeline fails because of missing environment variables, how do you fix it?

Short answer: First I find which variable is missing and where it was supposed to come from: the `environment` block, a credential, a parameter, the agent, or a previous stage. Then I fix the source — define it, fix the scope, or run the job once so parameters register. I also add a validation step early in the pipeline so it fails fast with a clear message.

Common causes

1. **Defined in the wrong scope** — set in one stage's `environment` block but used in another stage.
2. **`def` instead of `env.`** — `def TAG = ...` inside a `script` block is a local Groovy variable; `sh '$TAG'` can't see it. Use `env.TAG = ...`.
3. **Multibranch-only variables** — `BRANCH_NAME` and `CHANGE_ID` are empty in a normal Pipeline job.
4. **No `checkout scm`** — `GIT_COMMIT` and `GIT_BRANCH` are set by the Git checkout; on a new agent without checkout, they may be missing.
5. **Parameters on first run** — a new parameter is only registered after the first run of the updated Jenkinsfile.
6. **Credential binding** — the variable exists only inside the `withCredentials` block.
7. **Agent-specific variables** — set in one agent's node properties, but the build ran on another agent.
8. **Quoting** — `'$VAR'` lets the shell expand it; `"${VAR}"` makes Groovy expand it and fails if Groovy doesn't know it.

Example — fail fast with a clear message

```
stage('ValidateEnvironment') {
    steps {
        script {
            defrequired = ['AWS_REGION', 'IMAGE', 'TAG']
            defmissing = required.findAll { !env.getProperty(it) }
            if(missing) {
                error "Missing required environment variables:
${missing.join(', ')}"
            }
        }
    }
}
```

Hands-on experience: A pipeline failed with "invalid reference format" during `docker build`. The image tag was empty. A developer had changed `env.TAG = ...` to `defTAG = ...` while refactoring, so the shell never saw it. I fixed it and added the validation stage above, so the next mistake would show a clear message instead of a confusing Docker error.

> **Interview tip:** The `def` vs `env.` difference is a real-world detail that shows you've debugged pipelines yourself.

68. A job works in Dev but fails in Production. How do you troubleshoot it?

Short answer: If the same pipeline works in Dev but not in Prod, the code is probably fine and something in the environment is different. I compare the two environments side by side: credentials and permissions, configuration and secrets, network access, resource limits, the agent used, the image version, and cluster policies. I find the difference, fix it, and make environments more consistent.

What I compare

1. **The error itself** — console log and, for deployments, `kubectl describe pod` and `kubectl get events` in the prod namespace.
2. **Credentials and permissions** — prod uses different credentials; the IAM role or Kubernetes RBAC may be stricter.
3. **Configuration and secrets** — a missing ConfigMap key or secret in prod (`CreateContainerConfigError`).
4. **Network** — security groups, network policies, private endpoints, firewall rules; can the prod agent reach Nexus or the database?
5. **Resources** — namespace ResourceQuota, LimitRange, node capacity; pods stuck in `Pending` or `OOMKilled`.

6. **Policies** — Pod Security Admission, OPA/Kyverno policies that block privileged pods or `latest` tags.
7. **Versions** — same image tag? Same Helm chart version? Same Kubernetes version?
8. **Agent** — the prod deploy agent may have different tool versions (kubectl, Helm, Java).
9. **Data** — database schema or migrations differ between environments.

Hands-on experience: A deployment worked in Dev but pods failed in Prod with `CreateContainerConfigError`. `kubectl describe pod` showed a missing secret key. A new environment variable had been added to the Dev secret in Secrets Manager but not to Prod. I added it, the pods started, and we added a pre-deploy check comparing required secret keys across environments.

> **Interview tip:** Start with "the code is the same, so I look for what is different." It's a simple, logical opening that shows a calm troubleshooting mindset.

69. How do you handle rollback in a pipeline?

Short answer: Our rollback depends on the deployment method. With GitOps, we revert the image tag commit and ArgoCD deploys the previous version. With Helm, we use `helm rollback`. With plain Kubernetes, `kubectl rollout undo`. We also have a parameterized "Rollback" job that redeploys any previous image tag, and we automate rollback when health checks fail after a deploy.

Rollback methods

Method	Command / action
GitOps (ArgoCD)	`gitrevert <commit>` in the GitOps repo; ArgoCD syncs
Helm	`helm history orders -n prod` then `helm rollback orders <revision> -n prod`
Kubernetes Deployment	`kubectl rollout undo deployment/orders-service -n prod`
Blue-green	Switch the Service selector back to the old color
Redeploy a known tag	Parameterized Jenkins job with `IMAGE_TAG=<previous>`

Automatic rollback example

```
stage('Deploy Production') {
    steps {
        script {
            try{
                sh '''
                  helmupgrade --install orders ./charts/orders -n prod \
                    --setimage.tag=$TAG --wait --timeout 5m
                  ./smoke-test.sh prod
                '''
            } catch (err) {
                echo"Deployment or smoke test failed, rolling back"
                sh'helmrollback orders 0 -n prod --wait'
                error"Release ${env.TAG} rolled back: ${err}"
            }
        }
    }
}
```

- `--wait` makes Helm wait until pods are ready.
- `helm rollback orders 0` rolls back to the previous revision (0 means "previous").
- `error` still fails the build, so the team is alerted.
- Helm's `--atomic` flag (called `--rollback-on-failure` in Helm 4) can also roll back automatically if the upgrade fails.

Important points

- Rollback only works if old images still exist — keep them in the registry.
- **Database migrations** must be backward compatible, or rollback of the app breaks on the new schema.
- Practise rollback, so it's fast under pressure.

Hands-on experience: In my project, production rollback was a Git revert in the GitOps repo. During one incident, a new release had a memory leak. We reverted the commit, ArgoCD redeployed the previous version, and the service was stable again in about 4 minutes.

> **Interview tip:** Mention database compatibility. It's what separates a theoretical answer from a production-ready one.

70. If the version 1.2 deployment fails but version 1.1 is stable, what will you do?

Short answer: My first priority is to restore service, so I roll back to 1.1 immediately. Then I inform the team and stakeholders. After production is stable, I investigate 1.2 in a lower environment, find the root cause, fix it, and release a corrected version like 1.2.1 through the normal pipeline. Finally, we do a blameless post-mortem and add checks to catch this earlier.

Step by step

1. **Assess impact quickly** — is it a partial failure (new pods not ready, old pods still serving) or users affected?
2. **Roll back to 1.1** — GitOps revert, `helm rollback`, or redeploy tag 1.1. If the rolling update never completed, Kubernetes may still be serving 1.1 — I make sure the failed rollout is undone.
3. **Verify** — pods healthy, error rate and latency back to normal on Grafana, smoke tests pass.
4. **Communicate** — update the incident channel and stakeholders.
5. **Collect evidence** — pipeline logs, `kubectl describe`, pod logs, events, metrics from the failed attempt.
6. **Root cause** — config difference, missing secret, DB migration, dependency change, resource limits.
7. **Fix and test** — reproduce in staging, fix, release 1.2.1 through the full pipeline.
8. **Prevent** — add the missing test or check, consider canary releases, improve monitoring and alerts.

Hands-on experience: Version 1.2 of our orders service failed because new pods crashed on startup. The new version expected a config value that existed only in staging. I rolled back to 1.1 via a GitOps revert within minutes, confirmed metrics were normal, then added the missing value to production config and released 1.2.1 the next day. We added a startup config validation to catch it earlier.

> **Interview tip:** Always say "restore service first, investigate later". Interviewers check if you'd debug in production while users suffer.

71. What is a concurrency issue in pipelines, and how do you handle it?

Short answer: A concurrency issue happens when two pipeline runs work at the same time on something shared: the same environment, the same Git repo, the same workspace or the same image tag. For example, two deployments to Dev at once, or two

builds pushing to the GitOps repo and one fails with a push conflict. We prevent it with `disableConcurrentBuilds`, `lock` from the Lockable Resources plugin, milestone, unique image tags and retries for Git pushes.

Typical problems

- Two deployments to the same environment overlap — an older build finishes last and overwrites the newer version.
- Git push to the GitOps repo rejected because another build pushed first.
- Two builds share a workspace or a test database and break each other.
- The same mutable tag (`latest`) overwritten by parallel builds.

Solutions

Problem	Solution
Same job running twice	`options { disableConcurrentBuilds(abortPrevious: true) }`
Shared environment or resource across jobs	`lock(resource:'dev-environment') { ... }` (Lockable Resources plugin)
Older build deploying after a newer one	`milestone()` step before deploy
GitOps push conflict	`git pull --rebase` and retry
Same image tag	Unique tags (commit SHA or build number)

Example

```
pipeline {
    agent any
    options { disableConcurrentBuilds(abortPrevious: true) }
    stages {
        stage('Deploy to Dev') {
            steps {
                milestone(1)
                lock(resource: 'orders-dev-environment') {
                    sh './deploy.sh dev $TAG'
                }
            }
        }
        stage('Update GitOps') {
            steps {
```

```
                    retry(3) {
                        sh 'cd gitops && git pull --rebase origin main && git
  push origin main'
                    }
                }
            }
        }
    }
```

- `abortPrevious: true` cancels the older running build when a newer one starts.
- `milestone(1)` makes sure an older build can't pass this point after a newer build already has.
- `lock` lets only one build use the Dev environment at a time, even across different jobs.

Hands-on experience: When 10 services merged at the same time on release day, several builds failed pushing to the shared GitOps repo. I added `git pull --rebase` with `retry(3)`, and the failures stopped. We also locked the shared integration-test database with `lock`.

> **Interview tip:** Name the three tools — `disableConcurrentBuilds`, `lock` and `milestone` — and say which problem each one solves.

72. How do you set up and manage approvals before a production deployment?

(Source wording: "How do you calculate approvals before production?" — interpreted as how approvals are set up, who approves, and how many approvals are required.)

Short answer: We define who must approve and how many approvals are needed, based on the change risk. In Jenkins, the `input` step with `submitter` limits approval to the release managers group, and we can chain two `input` steps when two different teams must approve. In GitOps, approvals are pull request reviews on the production folder, with CODEOWNERS and a required number of reviewers. Every approval is recorded for audit.

How we decide the approvals

- **Normal release:** 1 approval from the release manager after QA sign-off.
- **High-risk change** (DB migration, infrastructure change): 2 approvals — for example, tech lead plus release manager.
- **Emergency fix:** 1 approval with a follow-up review after the release.
- **Pre-conditions before approval is even possible:** all tests green, Quality Gate passed, no CRITICAL vulnerabilities, change ticket approved.

Jenkins implementation (two approvers from different groups)

```
stage('Approvals') {
     agent none
     steps {
          timeout(time:1,unit:'DAYS') {
               script {
                    defqa=inputmessage: 'QA sign-off for production?',
                                   submitter: 'qa-leads', submitterParameter:
'APPROVER'
                    defrm=inputmessage: "Release ${env.TAG} to production?",
                                   submitter: 'release-managers',
submitterParameter: 'APPROVER'
                    echo"Approvedby QA: ${qa}, Release manager: ${rm}"
               }
          }
     }
}
```

Each `input` step is approved by one person, so chaining two steps with different `submitter` groups gives two independent approvals.

GitOps implementation

- Production manifests live in `envs/prod/`.
- `CODEOWNERS` assigns the release team to that folder.
- Branch protection requires, for example, 2 approving reviews and passing checks before merge.
- The merge itself is the approval record, and ArgoCD deploys after merge.

Hands-on experience: In my project, production releases needed QA sign-off plus release manager approval. In Jenkins we used the two chained `input` steps above, and the approvers' names were posted to Slack and attached to the change ticket. After moving production to GitOps, the same rule became "2 required reviews on `envs/prod/`".

> **Interview tip:** Talk about risk-based approvals and audit trails. That shows you understand approvals as a governance process, not just a button in Jenkins.